



POLSKO-JAPONSKA
AKADEMIA TECHNIK
KOMPUTEROWYCH

Wydział Informatyki

Katedra Systemów inteligentnych i Data Science

Inteligentne systemy przetwarzanie danych

Autor: Artur Krzowski

Nr albumu: s20522

**Zastosowanie modeli uczenia maszynowego do predykcji
niewydolności serca.**

Praca inżynierska

Promotor: Dr Thi Sinh Hoa Nguyen

Warszawa, luty, 2026

Streszczenie

Niniejsza praca porównuje skuteczność modeli uczenia maszynowego, dokładniej mówiąc lasów losowych (Random Forest), z wielowarstwowym perceptronem (MLP), w predykcji śmiertelności pacjentów dotkniętych niewydolnością serca. Badania zostały stworzone na podstawie bazy danych z 299 pacjentów, charakteryzującą się nierównowagą klas (32% zgonów i 68% przeżyć). Główne wyzwanie spowodował mały rozmiar danych, niebilansowanie klas oraz wyeliminowanie wycieku danych przez jedną z cech (time).

Aby poprawić jakość modelu, została zastosowana optymalizacja hiperparametrów i przygotowanie danych. Dla MLP przedstawiono regularyzację oraz balansowanie klas (undersampling, SMOTE). Ostatecznie lepszy wynik został osiągnięty przez Random Forest (F1-score: 0,680, Recall: 0,895) w porównaniu do MLP (F1-score: 0,638). Praca pokazuje, że na małych zbiorach, klasyczne algorytmy potrafią przewyższać sieci neuronowe. Ważne jest podkreślenie, że odpowiednie przygotowanie danych ma duże znaczenie, szczególnie dla MLP.

Słowa kluczowe: niewydolność serca, predykcja śmiertelności, uczenie maszynowe, lasy losowe (Random Forest), wielowarstwowym perceptron (MLP), niebilansowany zbiór danych, mały rozmiar danych, wyciek danych.

Abstract

This study compares the effectiveness of machine learning models, specifically Random Forests, with a Multilayer Perceptron (MLP) in predicting mortality among patients affected by heart failure. The research was conducted using a database of 299 patients, characterized by class imbalance (32% deaths and 68% survivors). The main challenges were the small dataset size, class imbalance, and the elimination of data leakage caused by one of the features (time).

To improve model quality, hyperparameter optimization and data preparation were applied. For the MLP, regularization and class balancing techniques (undersampling, SMOTE) were implemented. Ultimately, a better result was achieved by Random Forest (F1-score: 0.680, Recall: 0.895) compared to MLP (F1-score: 0.638). The work demonstrates that on small datasets, classical algorithms can outperform neural networks. It is important to emphasize that appropriate data preparation is highly significant, especially for MLP.

Key words: heart failure, mortality prediction, machine learning, Random Forest, Multilayer Perceptron (MLP), imbalanced dataset, small dataset size, data leakag.

Spis treści

Streszczenie	2
Abstract.....	2
1. Wstęp	5
2. Przegląd Literatury.....	7
2.1. Uczenie maszynowe w diagnostyce i predykcjach w kardiologii	7
2.2. Modele predykcyjne w niewydolności serca – stan badań.....	8
3. Modele uczenia maszynowego.....	10
3.1. Metody zespołowe	10
3.2. Bagging (Bootstrap Aggregating)	10
3.2.1. Random Forest (Lasy Losowe).....	11
3.2.2. Różnice między Random Forest a Bagging	12
3.4. Boosting.....	12
3.6. Wielowarstwowy Perceptron (MLP).....	14
3.6.1. Działanie neuronu.....	14
3.6.2. Warstwy neuronu.....	15
3.6.3. Proces uczenia: Propagacja wsteczna i funkcja straty	17
3.6.5. Funkcja aktywacji	17
3.6.4. Optymalizatory	18
3.7. Metody balansowania danych	20
3.7.1. Problem do rozwiązania	20
3.7.2. Konsekwencja niezbalansowania	20
3.7.3. Metody bilansowania:	21
3.7.4. Undersampling	21
3.7.5. Oversampling	22
3.7.6. SMOTE	22
4. Metodologia i środowisko badawcze	24
6.3. Cel i analiza wstępna	24
6.3. Eksploracyjna analiza Danych (EDA).....	25
6.3.3. Wykrycie i eliminacja wycieku informacji (Target leaking).....	26
6.3.3. Metryki oceny modeli.....	27
6.3.3. Środowisko pracy.....	28
5. Eksperymenty i wyniki	29
5.1. Eksperyment 1: Model Bazowy Random Forest.....	29
5.2. Eksperyment 2: Model Random Forest z Inżynierią cech	30
5.3 Eksperyment 3: Model sieci neuronowej na danych niezbalansowanych	31
5.4. Eksperyment 4: Model sieci neuronowej na danych zbalansowanych	35

6. Dyskusja i wnioski	37
6.1 Dyskusja wyników.....	37
6.2. Wnioski końcowe	37
6.3. Ograniczenia i kierunek rozwoju	38
Bibliografia	39

1. Wstęp

Niewydolność serca należy do chorób układu sercowo-naczyniowego, które szacunkowo odpowiadają za 32% wszystkich zgonów na świecie rocznie. Jest to około 17,9 miliona istnień. Występują one głównie z powodu zawałów serca i udarów mózgu z czego jedna trzecia z nich dotyka osoby poniżej 70-tego roku życia [1]. Choroby sercowo-naczyniowe są powiązane ze stylem życia, które prowadzimy, a konkretnie z czynnościami, które wpływają na nasz organizm takimi jak: niezdrowa dieta, mała aktywność fizyczna, papierosy, spożycie alkoholu. Rzeczy te prowadzą do nadciśnienia tętniczego, podwyższonego poziomu cukru we krwi czy otyłości. Choroby sercowo-naczyniowe oraz *niewydolności serca* są długoterminowe, jeśli nie są wykryte odpowiednio wcześniej będą się pogarszać z czasem. Współistnieją z objawami, po których można wywnioskować wystąpienie choroby, oto przykłady: duszności, ciągłe zmęczenie, obrzęki kończyn, zawroty głowy, a nawet utrata przytomności. Na *niewydolność serca* choruje obecnie 64 miliony osób na świecie, co sprawia, że jest istotnym problemem w kontekście zdrowia na świecie [2]. Szacuje się, że prawdopodobieństwo wystąpienia tej choroby to między 20% a 25% [3]. Może mieć na to wpływ ogólne starzenie się społeczeństw (dzięki technologii wydłużamy czas życia). Choroby związane z *niewydolnością serca* powodują obciążenie w kontekście wydatków państwa na opiekę zdrowotną. Składa się na to długotrwałe leczenie, częste wizyty, badania, hospitalizacja. Najwięcej pieniędzy nie idzie więc na leki, ale na pobyt w szpitalu, aby wyciszyć konsekwencję którą wywołała choroba serca [4]. Ostatecznie nie da się wskazać jednej przyczyny, gdyż czynników jest dużo oraz u każdego pacjenta choroba może przebiegać trochę inaczej. Lekarz ma problem z analizą tak wielu rzeczy na raz, co utrudnia wczesne zdiagnozowanie oraz stwierdzenie kto jest zagrożony.

Dzięki rozwojowi technologii oraz przeniesieniu zapisu z papieru do komputera, dokumentacja medyczna jest w jedynym miejscu, co powoduje, że można ją analizować i wyciągać ciekawe wnioski. Dane te są pokaźne, złożone, posiadają dużą liczbę zmiennych oraz nieliniowych zależności, co sprawia, że metody nauczania maszynowego dobrze sprawdzają się w zadaniach predykcyjnych. Problem badawczy który podejmuje w tej pracy, to czy na podstawie standardowych danych klinicznych jest możliwe skuteczne rozpoznanie, którzy pacjenci są najbardziej narażeni na zgon i jakie parametry najbardziej na to wpływają podczas okresu obserwacji. Rzeczą, która zmotywowała mnie do napisania tej pracy, jest zastosowanie opracowanych modeli by wspomóc proces podejmowania decyzji przez lekarzy. Model predykcyjny, który przy odpowiednim dostrojeniu parametrów byłby w stanie z wysoką precyzją określać pacjenta jako osobę wysoko narażoną stanowiłoby nie tylko wsparcie dla osób pracujących w służbie zdrowia, ale także odciążałoby skarb państwa. Pieniądze mogłyby dofinansować inne badania, które polepszyłyby stan życia ludzi, a dzięki zwolnionych miejscach w szpitalach oraz przychodniach, można byłoby przyjmować i diagnozować ludzi z innymi schorzeniami.

Głównym celem mojej pracy i jest porównanie dwóch modeli *uczenia maszynowego* pod kątem *predykcji śmiertelności* pacjentów z *niewydolnością serca*. Analizie podano model Random Forest, którego mechanizm jest oparty na zespołach drzew decyzyjnych, oraz model sztucznej sieci neuronowej, który charakteryzuje się większą elastycznością i wyższą produkcją, jeśli chodzi o nieliniowe zależności w danych.

Oto moje pytania badawcze:

1. Czy bardziej złożony model (*MLP*) jest w stanie osiągnąć lepsze wyniki niż prostszy (*Random Forest*) na tym konkretnym zbiorze danych?
2. Jakie techniki inżynierii cech (np. dyskretyzacja, interakcje) wpływają na wydajność modelu?
3. Jak poszczególne hiper parametry sieci neuronowej (architektura, regularyzacja) wpływają na jej zdolność predykcyjną?

Zakres pracy obejmuje analizę jednego zbioru danych, które dotyczy pacjentów z *niewydolnością serca*, przygotowanie tych danych do analizy, przeprowadzenie ekspertów w celu uzyskania jak najlepszych wyników oraz porównanie otrzymanych danych.

Praca podzielona jest na kilka rozdziałów, które stanowią kolejne etapy realizacji badań. *Rozdział drugi* przedstawia przegląd literatury, która dotyczy się *niewydolności serca* oraz w jaki sposób metody uczenia maszynowego mogą pomóc w analizie danych medycznych. W *Rozdziale trzecim* przedstawione są modele uczenia maszynowego które zostały użyte w pracy. *Rozdział czwarty* został poświęcony opisowi zbioru danych oraz wybranej przeze mnie metodologii badawczej, obejmującej przygotowanie danych do użytku, inżynierii cech oraz budowania modeli predykcyjnych. W *rozdziale piątym* pokazano wyniki eksperymentów oraz porównano ze sobą oba modele. *Ostatni rozdział* posiada Dyskusję na temat wszystkiego co zrobiono oraz wnioski końcowe. Zawarto w nim też możliwe kierunki ekspansji tego tematu.

2. Przegląd Literatury

W tym rozdziale zostaną przedstawione podstawy teoretyczne i badawcze pracy. Zostanie omówiony przebieg oraz aktualne zastosowanie *uczenia maszynowego (ML)* w kardiologii, z podkreśleniem predykcji *niewydolności serca*. Zostaną również opisane modele tworzące rdzeń badań: *Random Forest* oraz wielowarstwowy perceptron (*MLP*), i ich kluczowe komponenty. Niniejsza praca podejmuje próbę wypełnienia luki badawczej zidentyfikowanej w badaniu, na którym opieram swoje opracowanie, polegającej na porównaniu klasycznego i głębokiego *uczenia* na narzuconym (i ograniczonym) zbiorze danych.

2.1. Uczenie maszynowe w diagnostyce i predykcjach w kardiologii

Metody *uczenia maszynowego* znajdują coraz szersze zastosowanie w diagnostyce, pomagając lekarzom w analizowaniu obszernych i złożonych danych klinicznych oraz wpływając na podejmowane przez nich decyzje. Kardiologia to dziedzina, w której wykorzystywane są różne typy danych takie jak: sygnały elektrokardiograficzne, obrazy diagnostyczne, dane kliniczne opisujące stan zdrowia pacjenta. Wykorzystanie algorytmów *uczenia maszynowego* umożliwia automatyzacja analizy oraz identyfikację wzorców, które mogą stanowić problem dla lekarza.

Najbardziej rozpowszechnioną praktyką w kardiologii jest automatyczny zapis analizy elektrokardiografii (EKG). Algorytmy *uczenia maszynowego*, w szczególności te które są oparte na *MLP*, Są już teraz wykorzystywane, aby wykryć zaburzenia rytmu serca (np. migotanie przedsionków), jak i również do klasyfikacji sygnałów EKG pod kątem wykrycia jakichś nieprawidłowości. Badania pokazują, że systemy które opiera się na *uczeniu maszynowym* potrafią obciążać wyniki skuteczności porównywalne do doświadczonego specjalisty [5].

Kolejny obszar, w którym wykorzystuje się *ML*, to analizy obrazów medycznych, w tym segmentacja obrazów rezonansu magnetycznego oraz tomografii komputerowej serca. Modele głębokiego *uczenia maszynowego* umożliwiają wyodrębnienie struktur anatomicznych takich jak komory serca, co wspiera oceny funkcji skurczowej i objętościowej serca. Automatyzacja tego procesu jest w stanie przyczynić się do skrócenia czasu analizy badań obrazowych [6].

Mechanizm *uczenia maszynowego* znajdują również zastosowanie w obszarze schorzeń sercowo-naczyniowych, takich jak zawał serca czy zgon pacjenta. Wykorzystują dane demograficzne, kliniczne oraz laboratoryjne do predykcji osób znajdujących się w grupie podwyższonego ryzyka, co ma duże znaczenie w diagnostyce i leczeniu [7].

Mimo tego, że zarówno informatyka jak i modele *uczenia maszynowego* dynamicznie się rozwijają, nadal powinny być stosowane jako narzędzia wspomagające a nie zastępujące lekarzy.

2.2. Modele predykcyjne w niewydolności serca – stan badań

Duże miejsce w literaturze dotyczącej metod *uczenia maszynowego* w zakresie *niewydolności serca* stanowią prace, które skupiają się na predykcji pod kontem przeżywalności pacjentów na podstawie danych klinicznych. Badanie Chicco i Jurman [8] z 2020 roku jest klasycznym przykładem podejścia analizującego efektywność modeli działających w obszarze predykcji. Moim zdaniem stanowi ważny punkt odniesienia dla mojej pracy.

W badaniu autorzy zaprezentowali analizę możliwości predykcyjnych przeżycia pacjentów z *niewydolnością serca*, kiedy również bazowali na dość *małym zbiorze* pacjentów (299 pacjentów) oraz ograniczonym zestawie cech klinicznych. Główny cel badania to stwierdzenie, czy proste parametry *stężenie kreatyniny w surowicy* i *frakcja wyrzutowa lewej komory serca*, mogą być wystraczające do stworzenia modelu predykcyjnego z akceptowalną skutecznością predykcyjną.

Autorzy wykorzystali modele *ML*, między innymi: drzew decyzyjnych, *Random Forest*, *XGBoost* i prostych sieci neuronowych. Z wyników eksperymentu można wywnioskować, że modele które opierały się na algorytmach zespołowych i drzewiastych (takie jak: *Random Fores* i *Gradient Boosting Machines*) osiągały najwyższe wyniki predykcyjne. W badaniu wartości miar jakości modeli świadczyły, że predykcja przeżycia na podstawie 2 cech może być efektywna a nawet klinicznie użyteczna [8]. Autorzy podkreślają, że mimo ograniczonego zestawu danych, odpowiednio dobrane i dostrojone modele *uczenia maszynowego* są w stanie znaleźć istotne zależności, aby predykcja była zadowalająca, co otwiera drogę *uczenia maszynowego* w analizie danych klinicznych na wysokim poziomie.

Inne publikacje także posiadają metody *uczenia maszynowego*, analizujące przeżycie pacjentów z *niewydolnością serca*. W pracy autora Krittanawong et z 2017 roku opisano zastosowanie *ML*, takich jak *Random Forest* i *Support Vector Machine (SVM)*. Użyto ich do predykcji wyników klinicznych, które uwzględniają dane z rejestrów medycznych osób chorujących na choroby sercowo-naczyniowe (w tym HF (*heart failure*)) [7]. Autorzy w badaniu podkreślają jednak, że skuteczność modeli zależy głównie od danych, którymi karmione są modele, wyboru cech oraz metod walidacji.

Przegląd literatury wskazuję, że głównym wyzwaniem w analizie danych dotyczących *niewydolności serca*, jest ograniczona liczba uznanych cech prognostycznych. Mimo tego, badania takie jak Chicco and Jurman (2020 rok) pokazują, że niewielki zestaw cech jest w stanie zbudować skuteczny model, ale ważne jest, aby pamiętać, że model działał dobrze na danych, na których był testowany, nie ma pewności, że będzie działała tak samo dobrze na innych danych oraz że zanim się go wdroży należy sprawdzić go na zupełnie nowych i niezależnych zbiorach. Dotychczasowe badania obrazują, że *uczenie maszynowe* ma

potencjał w zadaniach predykcyjnych w obszarze *predykcji śmiertelności* pacjentów z *niewydolnością serca* (i to proste jak i bardziej złożone modele).

3. Modele uczenia maszynowego

Tą część pracy zostanie poświęcone na przedstawienie zasad działania dwóch kluczowych algorytmów uczenia maszynowego: *Random Forest* (Lasy Losowe) oraz wielowarstwowego perceptronu (MLP).

3.1. Metody zespołowe

Metody zespołowe (*Ensemble Methods*) to techniki uczenia maszynowego, łączące predykcje kilku modeli bazowych, aby uzyskać lepszą, stabilniejszą oraz dokładniejszą predykcję, niż którykolwiek z modeli mógłby osiągnąć sam. Głównym założeniem tego sposobu jest koncept „mądrość tłumu”. Opiera się on na założeniu, że opinia grupy jest trafniejsza niż opinia jednego eksperta. W uczeniu maszynowym, błędy poszczególnych modeli, mogą się wzajemnie znosić, dzięki połączeniu zróżnicowanych modeli, co przyczynia się do poprawy wydajności.

Najpopularniejsze metody zespołowe to: *Bagging* oraz *Boosting*

3.2. Bagging (Bootstrap Aggregating)

Bagging to skrót od *Bootstrap Aggregating*. Jest jedną z największych i najprostszych, jednocześnie skutecznych metod zespołowych. Wprowadził ją Leo Breimana w 1996 roku [15]. Jego głównym celem jest zmniejszenie wariancji modeli predykcyjnego, dzięki czemu ma większą stabilność i odporność na przeuczenia.

Algorytm *Bagging* bazuje na tworzeniu wielu (T) niezależnych zbiorów danych treningowych poprzez losowanie ze zwracaniem z oryginalnego zbioru treningowego (D). Każdy nowy zbiór, nazywany z próbkami bootstrap, ma jednakowy rozmiar N jak oryginalne zbiór D . Losowani ze zwracaniem oznacza, że niektóre obserwacje z D mogą pojawić się w danej próbce niejednokrotnie, natomiast inne mogą nie zostać wylosowane w ogóle. Według szacunków każda próbka bootstrap ma około 63,2% z unikalnych obserwacji z oryginalnego zbioru, natomiast 36,8% obserwacji tworzy zbiór *out-of-bag* (OOB).

Na każdej z T próbek trenowane jest niezależny model bazowy. W związku z tym, a że każdy model uczy się na nieco innych danych, modele są zróżnicowane. Ostateczną predykcję uzyskuje się dzięki agregacji wyników wszystkich T modeli:

- **Wobec klasyfikacji:** Końcowa decyzja podejmowana jest za pomocą głosowania większości (jeden model posiada jeden głos i wybrana zostaje klasa z największą ilością głosów).
- **Wobec regresji:** ostateczna predykcja jest średnią arytmetyczną predykcji wszystkich modeli.

Schemat algorytmu:

Krok 1: Dla $t = 1$ do T :

- a. Krok 1.1: Wygenerowanie próbki *bootstrap* D_t o rozmiarze N poprzez losowanie ze zwracaniem z oryginalnego zbioru danych D .
- b. Krok 1.2: Wytrenowanie modeli bazowy h_t na próbce D_t .

Krok 2: Agreguj predykcje modeli $h_1, \dots, h_T$:

- Klasyfikacja:

$$\hat{H}(x) = \text{mode} \{h_1(x), \dots, h_T(x)\}$$

- Regresja:

$$\hat{H}(x) = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

Out-of-bag to wbudowany mechanizm walidacyjny w *Bagging*. W związku z tym, że pewna część obserwacji, które wykorzystuje algorytm nie przyczynia się do trenowania modelu, dlatego nazywa się je *Out-of-bag*. Jednak ten mechanizm ma ważną rolę, gdyż można go wykozystać, do oceny wydajności modelu i nie trzeba tworzyć osobnego zbioru walidacyjnego. Do każdej obserwacji z oryginalnego zbioru danych, jest się w stanie uzyskać protekcję, uśredniając wynik dla modeli, które nie wykorzystywały tej obserwacji w procesie uczenia. Błąd szacowany na *OOB* jest oszacowaniem błędu generalizacji, który nie jest systematycznie zawyżony ani zaniżony.

3.2.1. Random Forest (Lasy Losowe)

Random Forest to jeden z modeli *uczenia maszynowego* który jest zarówno prosty jak i skuteczny. Klasyfikuje się do zespołowych (*ensemble*) algorytmów (należy do rodziny *Bagging*). Wprowadzony w 2001 roku przez Leo Breimana [9], jest rozszerzeniem metody *Bagging* ale dodatkowo wprowadza losowość na poziomie wyboru cech w każdym węźle drzewa decyzyjnego. Skutkuje to zwiększeniem zróżnicowania modeli bazowych i lepszą wydajnością.

Schemat algorytmu:

Krok 1: Dla $t=1$ do T (liczba drzew w lesie):

Krok 1.1: Wygenerowanie próbki *Bootstrap* D_t o rozmiarze N poprzez losowanie ze zwracaniem z oryginalnego zbioru danych D .

Krok 1.2: Zbuduj drzewo decyzyjne h_t na próbce D_t . W każdym węźle drzewa, podczas poszukiwania najlepszego podziału:

Krok 1.2.1: Wybrać losowy podzbiór m cech spośród wszystkich M dostępnych cech (gdzie $m \ll M$, $m = \sqrt{M}$ lub $m = \log_2(M)$).

Krok 1.2.2: Znaleźć najlepszy podział, używając tylko cech z losowo wybranego podzbioru.

Krok 2: Agregowanie predykcji T drzew:

- Klasyfikacja:

$$\hat{H}(x) = \text{mode}\{h_1(x), \dots, h_T(x)\}$$

- Regresja:

$$\hat{H}(x) = (1/T) \sum_{t=1}^T h_t(x)$$

3.2.2. Różnice między Random Forest a Bagging

Różnica między *Bagging* a *Random Forest* jest krok 1.b.i. W *Baggingu*, każde drzewo jest budowane z uwzględnieniem wszystkich dostępnych cech. W *Random Forest* algorytm ma do dyspozycji tylko losowy podzbiór cech. Warstwa losowości ma kluczowe znaczenie: zapobiega ona sytuacji, w których jedna bardzo silna cecha prognostyczna dominowałaby w procesie budowy wszystkich drzew, co prowadziłoby do ich skorelowania. Dzięki wyborowi cech, drzewa w lesie są bardziej zróżnicowane, co z reguły prowadzi do zmniejszenia wariancji całego zespołu i poprawy jego zdolności do generalizacji.

Leo Breiman pokazał, że Błąd generalizacji losu losowego zbiega do granicy, gdy liczba drzew rośnie do nieskończoności. Granica zależy od:

1. Siły pojedynczych drzew w lesie: Im niższy błąd pojedynczych drzew, tym niższy błąd całego lasu.
2. Korelacji między drzewami: Im niższa korelacja między drzewami, tym niższy błąd lasu.

Random Forest dzięki stosowaniu Baggingu oraz losowego wyboru cech, idzie w stronę budowania silnych i nieskorelowanych ze sobą drzew, dzięki czemu model ma wysoką dokładność i jest odporny na przeuczenia.

3.4. Boosting

To druga metoda zespołowa. W przeciwieństwie do Baggingu, buduje model w sposób sekwencyjny (a nie równoległy). Jego głównym założeniem, jest iteracyjne poprawianie modelu, za pomocą skupiania się na błędach popełnionych we wcześniejszych krokach. Każdy kolejny model jest uczony w taki sposób, żeby korygować błędy poprzednika.

Proces modelu zaczyna się od wytrenowania prostego modelu bazowego na oryginalnym zbiorze danych. Później, w każdej następnej iteracji, algorytm analizuje błędy popełnione przez dotychczasowy zbiór modeli. Błędnie sklasyfikowane obserwacje przypisywane są większe wagi, dzięki czemu stają się „ważniejsze” podczas uczenia kolejnego modelu. Przez to, kolejny model zmuszony jest na skupieniu się na trudniejszych przypadkach, będąc jak problematycznymi dla poprzednich modeli.

Proces powtarza się przez z góry określoną liczbę iteracji (T). Końcowa predykcja jest ważoną z sumą predykcji wszystkich modeli w grupie, a wagi poszczególnych modeli zależą od ich skuteczności, oznacza to, że lepsze modele mają większy wpływ na ostateczną predykcję.

Schemat algorytmu Boosting:

Krok 1: Inicjowanie wag dla wszystkich obserwacji w zbiorze treningowym

Krok 2: Dla $t=1$ do T :

a. Krok 2.1: Trenowanie modeli bazowy h_t na zbiorze treningowym z uwzględnieniem wag obserwacji.

Krok 2.2: Obliczanie błędu modelu h_t .

Krok 2.3: Obliczanie wagi modelu α_t na podstawie jego błędu (im mniejszy błąd, tym większa waga).

Krok 2.4: Zaktualizowanie wagi obserwacji: zwiększenie wagi dla obserwacji błędnie sklasyfikowanych przez h_t .

Krok 3: Ostateczna predykcja to ważona suma predykcji wszystkich modeli, schemat przedstawia algorytm dla zadania klasyfikacji binarnej:

$$\hat{H}(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

Ale można go też uogólnić dla klasyfikacji wieloklasowej, wtedy wygląda tak:

$$\hat{H}(x) = \arg \max_{k \in \{1, \dots, K\}} \sum_{t=1}^T \alpha_t \cdot I(h_t(x) = k)$$

Najpopularniejsze Algorytmy z rodziny *Boosting* są: *AdaBoost* (Adaptive boosting)[16] i *Gradient Boosting Machines (GBM)* [17]. *Gradient Boosting* jest rozszerzeniem *AdaBoost* i opiera się na iteracyjnym minimalizowaniu funkcji straty przy użyciu dodawania modeli, które najlepiej "wyjaśniają" reszty (błędy) z poprzedniego kroku. Najbardziej popularne wersje *Gradient Boostingu*, takie jak *XGBoost*, *LightGBM*, oraz *CatBoost* [18], są obecnie uważane za niektóre z najlepszych i najbardziej używanych metody w konkursach uczenia maszynowego oraz praktycznej aplikacji.

Boosting, w przeciwieństwie do *Bagging*, nie koncentruje się na redukcji wariancji, ale redukcji obciążenia (*bias*) samego modelu. *Boosting* jest bardziej podatny na przeuczenie niż *Bagging*, zwłaszcza jeśli liczba iteracji jest zbyt duża lub modele bazowe są zbyt złożone. Poprzez sekwencyjne dodawanie modeli korygujących błędy, zespół jest w stanie nauczyć się bardziej złożonych zależności i osiągnąć wyższą złożoność.

3.6. Wielowarstwowy Perceptron (MLP)

Wielowarstwowy Perceptron (Multi-Layer Perceptron, MLP) jest podstawą w rozmowach a sztucznej inteligencji i sieciach neuronowych. W odróżnieniu od Random Forest, działa w pojedynkę, ale jako głęboko hierarchiczna i nieliniowa funkcja manipulująca wejście na wyjście, czyli innymi słowy, przekształca dane wejściowe (np. cechy), w wynik końcowy ((np. klasę, prawdopodobieństwo czy wartość liczbową).

Mamy dwa główne typy architektur sieci neuronowych: płytkie i głębokie. Płytke sieci, takie jak klasyczny MLP, zwykle mają jedną albo dwie warstwy ukryte. Głębokie sieci, czyli Deep Learning, składają się z wielu warstw ukrytych, dzięki czemu lepiej radzą sobie z bardziej złożonymi i hierarchicznymi zależnościami w danych. W tej pracy, biorąc pod uwagę charakter i wielkość zbioru danych, skupiam się na modelu płytkiej sieci neuronowej.

3.6.1. Działanie neuronu

MLP trochę przypomina to, co dzieje się w naszym mózgu. Podstawową jednostką jest tu neuron, oczywiście sztuczny, ale wzorowany na biologicznym. To po prostu matematyczny model komórki nerwowej. Cały proces działania takiego neuronu można rozłożyć na dwa kroki:

- **Kombinacja liniowa:** Neuron oblicza ważoną sumę wszystkich „n” sygnałów wejściowych x_i (pochodzących od innych neuronów lub danych), do której dodawany jest składnik stały, tzw. *bias* (b). Wynik to tzw. pobudzenie/aktywacja (*activation*) z :

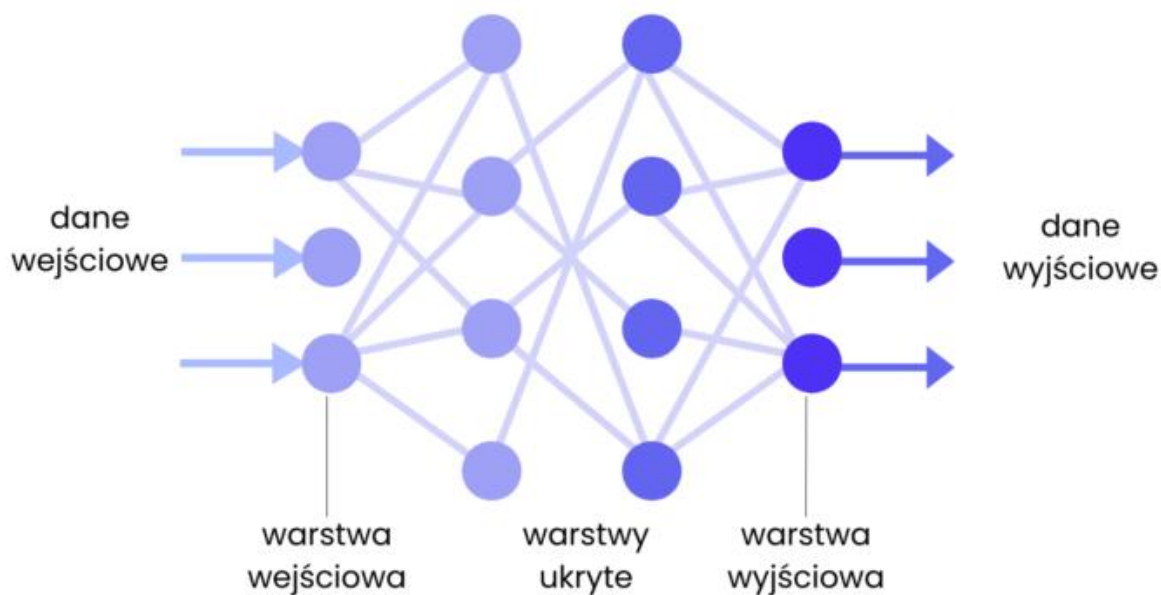
$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

gdzie w_i to waga przypisana i -temu wejściu.

- **Funkcja aktywacji:** Na zawartość z nakładana jest nieliniowa funkcja aktywacji, która decyduje o tym, czy i jak silnie neuron aktywuje się. To kluczowy element, gdyż wprowadza nieliniowość, która rozróżnia cieć neuronową od prostych uczeń maszynowych bazujących na linowym rozgraniczaniu. W warstwach ukrytych, w obecnych czasach, używa się funkcji *ReLU* (*Rectified Linear Unit*):

$$f(z) = \max(0, z) ,$$

która jest prosta obliczeniowo i skutecznie łagodzi problem zanikających gradientów [10].



Źródło: opracowanie własne

3.6.2. Warstwy neuronu

Neurony są segregowane w warstwy, co oznacza, że neurony nie działają niezależnie i każda warstwa pełni określona rolę. Tworzą graf, czyli zbiór węzłów (neurony) i połączeń (wagi). Rozróżnia się 3 warstwy:

- **Warstwa wejściowa:** to pierwsza warstwa, która ma bezpośredni kontakt z danymi. Neurony w warstwie wejściowej nie wykonują operacji matematycznych takich jak sumy ważone czy funkcje aktywacji oraz nie modyfikują danych, a jedynie przekazują je dalej w postaci wektora cech x . Wektor cech przyjmuje się w postaci:

$$x = [x_1, x_2, \dots, x_n]$$

Liczba neuronów odpowiada liczbie cech.

- **Warstwy ukryte:** znajdują się między warstwą wejściową a wyjściową. Sieć może zawierać jedną lub wiele warstw ukrytych - im więcej warstw, tym bardziej złożone zależności sieć może modelować. W tej warstwie wykonywane są właściwe obliczenia, polegające na liniowej kombinacji wejść z wagami i zastosowaniu nieliniowej funkcji aktywacji. Matematycznie, przejście przez l -tą warstwę ukrytą można opisać jako:

1. **Kombinacja liniowa:** Dla każdego neuronu i w warstwie l obliczana jest ważona suma sygnałów wejściowych z poprzedniej warstwy:

$$(z_i^{[l]} = \sum_{j=1}^{n^{[l-1]}} w_{ij}^{[l]} a_j^{[l-1]} + b_i^{[l]})$$

gdzie: $w_{ij}^{[l]}$ - waga łącząca neuron j z warstwy poprzedniej z neuronem i w bieżącej warstwie, $a_j^{[l-1]}$ - aktywacja neuronu j z warstwy poprzedniej, $b_i^{[l]}$ - bias (przesunięcie) neuronu i w warstwie l , $n^{[l-1]}$ - liczba neuronów w warstwie poprzedniej.

2. **Funkcja aktywacji:** Na wynik kombinacji liniowej nakładana jest nieliniowa funkcja aktywacji:

$$(a_i^{[l]} = f^{[l]}(z_i^{[l]})),$$

$$(z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]})$$

Gdzie $f^{[l]}$ to funkcja aktywacji (np. *ReLU*, *sigmoidalna*, *tanh*), – macierz wag o wymiarach $(n^{[l]} \times n^{[l-1]})$, $a^{[l-1]}$ - wektor aktywacji z warstwy poprzedniej, $b^{[l]}$ - wektor biasów,

- **Warstwa wyjściowa:** generuje ostateczną predykcję sieci. Liczba neuronów zależy od zadania: przy klasyfikacji binarnej stosuje się jeden neuron z funkcją aktywacji sigmoidalnej:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

gdzie wyniki interpretuje się jako prawdopodobieństwo $P(\text{klasa}=1|x)$, której wynik interpretuje się jako prawdopodobieństwo przynależności do klasy pozytywnej. Dla klasyfikacji wieloklasowej (gdzie liczba klas $K \geq 2$), warstwa wyjściowa zawiera K neuronów i używa się funkcji *Softmax*:

$$\text{Softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

gdzie: z_k – wartość wyjściowa (logit) dla k -tej klasy, K – całkowita liczba klas, mianownik - suma wykładniczych wartości wszystkich logitów, która przekształca wektor wartości wyjściowych (logitów) w rozkład prawdopodobieństwa nad wszystkimi klasami.

Rozróżnienie *sieci neuronowych* na płytkie i głębokie jest ważne w kontekście zrozumienia mocy oraz złożoności modelu. Podział opiera się na liczbie warstw ukrytych w sieci.

- **Sieci płytkie (Shallow Neural Networks):**

1. Są to sieci neuronowe z jedną warstwą ukrytą.
2. Uniwersalne Twierdzenie o Aproksymacji mówi, że jeśli masz sieć neuronową z jedną warstwą ukrytą i nieliniową funkcją aktywacji, możesz przybliżyć dowolną ciągłą funkcję tak dokładnie, jak chcesz. Wystarczy, że ta warstwa ukryta będzie miała odpowiednio dużo neuronów.
3. Ograniczenia: W teorii wszystko wygląda dobrze, ale w praktyce, żeby naprawdę dobrze przybliżyć skomplikowane funkcje, czasem trzeba wrzucić

do jednej warstwy tak dużo neuronów, że robi się to zupełnie niepraktyczne. Sieci płytkie po prostu słabo radzą sobie z nauką złożonych, wielopoziomowych reprezentacji danych.

- **Sieci głębokie (Deep Neural Networks - DNN):**

1. Sieci neuronowe z dwiema lub więcej warstwami ukrytymi.
2. Zalety: Głębokie sieci mają tu sporo do powiedzenia. Uczą się cech krok po kroku - najpierw wyłapują proste rzeczy, jak krawędzie na zdjęciu, potem łączą je w coraz bardziej złożone kształty i obiekty. Dzięki temu hierarchicznemu podejściu wyciągają z danych dużo więcej, robiąc to szybciej i dokładniej. W praktyce osiągają lepsze wyniki, zużywając mniej neuronów niż te prostsze, płytkie sieci.
3. Zastosowania: Głębokie sieci zrewolucjonizowały takie dziedziny jak rozpoznawanie obrazów, przetwarzanie języka naturalnego i rozpoznawanie mowy.

3.6.3. Proces uczenia: Propagacja wsteczna i funkcja straty

Uczenie MLP to zmniejszenie do minimum funkcji straty. Używa się do tego propagacji wstecznej (*backpropagation*) oraz metod optymalizacji gradientowej [15]. W projekcie użyto binarnej entropii krzyżowej jako funkcję straty:

$$L = -\frac{1}{m} \sum_{j=1}^m [y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)]$$

Gdzie m to liczba próbek, y_j to prawdziwa etykieta (0 lub 1), a $\hat{y}_j$ to przewidywane prawdopodobieństwo dla j -tej próbki. Do aktualizacji parametrów: wag oraz biasów, użyto algorytmu *Adam* [13]. W celu unikania przeuczenia, korzystano z *Dropout* [14].

3.6.5. Funkcja aktywacji

Jest to kluczowy element każdego neuronu w sieci. Powodują one nieliniowość modelu, co umożliwia uczenie się złożonych wzorców. Podstawową relację między wejściem a wyjściem opisuje wzór:

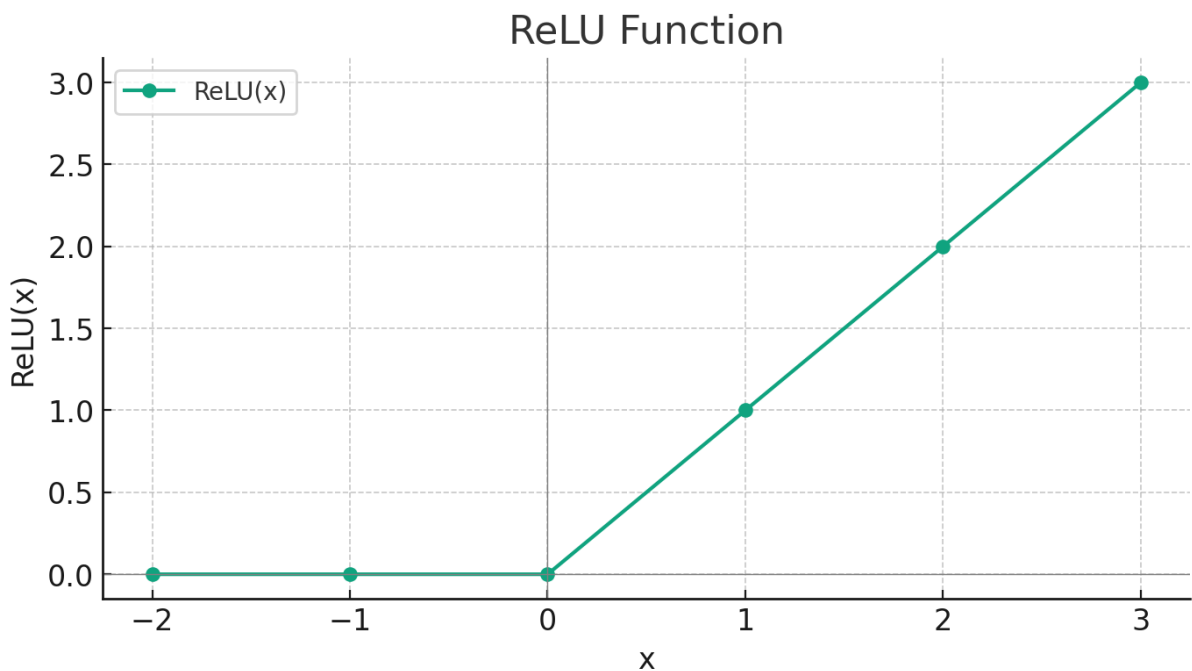
$$a = f(z) = f\left(\sum_i (w_i x_i) + b\right)$$

gdzie z to pobudzenie neuronu (liniowa kombinacja wejść), f to funkcja aktywacji, a to aktywacja (wyjście neuronu).

W eksperymentach użyto funkcję ReLu (Rectified Linear Unit) [10]:

$$f(z) = \max(0, z)$$

Która jest najpopularniejszą opcją, ze względu na efektywność obliczeniową oraz zmniejszanie problemu znikających gradientów.



Źródło: opracowanie własne

3.6.4. Optymalizatory

Proces uczenia sieci neuronowej opiera się na minimalizacji funkcji straty $L(\theta)$ z każdą iteracją, względem parametrów modelu θ (wag i biasów). Polega on na rozwiązaniu problemu optymalizacyjnego:

$$\min_{\theta} L(\theta)$$

Najprostszą metodą jest algorytm gradientu prostego (Gradient Descent, GD), który dla parametru „ θ ” w kroku „ t ” dokonuje aktualizacji:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Gdzie η to współczynnik uczenia, a $\nabla L(\theta_t)$ to gradient funkcji straty obliczony dla całego zbioru treningowego. Dla tego zbioru treningowego o liczbie próbek N , gradient definiuje się jako:

$$\nabla_{\theta} L(\theta_t) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} l(x_i, y_i; \theta_t)$$

Gdzie $l(x_i, y_i; \theta_t)$ to wartość funkcji straty dla pojedynczej próbki treningowej (x_i, y_i) , przy parametrach θ_t . Warto pamiętać, że klasyczny GD ma wady: obliczenie gradientu dla całego zbioru danych odbywa się w powolny sposób i wymaga dużej ilości pamięci, a dobranie właściwej wartości „ η ” jest trudny (zbyt duża wartość powoduje omijanie minimum, a za mała do bardzo wolnego „przeszukiwania”).

Lepszym rozwiązaniem jest postawienie na adaptacyjny optymalizator zwany *Adam* (*Adaptive Moment Estimation*). Zaprojektowany przez Kingmę i BA w 2014 roku [12]. Łączy on w sobie zalety dwój innych zaawansowanych metod: Momentum (dodaje bezwładność, aby przyspieszyć ruch w kierunku zgodnym z wcześniejszymi krokami) oraz RMSprop (które adaptacyjnie skaluje krok uczenia dla każdego parametru osobno, na podstawie historycznych wartości gradientu). Działa taki sposób:

Krok 1: Estymacja pierwszego momentu (średniej gradientów) „ m_t ”:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

(gdzie $g_t = \nabla L(\theta_t)$). To nadaje algorytmowi "pęd".

Krok 2: Estymacja drugiego momentu (nieobciążonej wariancji gradientów) v_t :

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

To mierzy zmienność gradientu w czasie.

Krok 3: Korekcja biasu (ważna na początkowych krokach):

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Krok 4: Aktualizacja parametrów:

$$\theta_{t+1} = \theta_t - \frac{\eta \widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

Domyślne wartości hiperparametrów to $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Ważną zaletą *Adam* jest to, że dla każdego parametru obliczane jest indywidualne tempo uczenia. Oznacza to, że jeśli jakiś parametr występuje rzadko, to dostaje większą aktualizację, natomiast te o niższych gradientach mniejsze. Dzięki temu „Adam” jest odporny na dobór współczynnika η oraz skuteczny w szerokim zakresie problemów.

Trzeba też poruszyć problem *przeuczenia* (*overfitting*). Problem polega na tym, że jeśli model perfekcyjnie dostosuje się („nauczy się”) do danych treningowych jest w stanie zignorować na nich szum, ale traci zdolność generalizacji. Oznacza to, że będzie niepoprawnie działał na nowych danych.

Aby przeciwdziałać przeuczeniu oraz aby sprawić, aby sieć uczyła się bardziej ogólnych reprezentacji, stosuje się technikę regularyzacji. Jedną z najlepszych, jest *Dropout*, stworzony przez Srivastavę, Hintona i innych w 2014 roku [13]. Polega ona na tym, że podczas każdej iteracji treningowej, każdy neuron (poza tymi w warstwie wejściowej), ma określone prawdopodobieństwo „p” (np. 0,5) bycia tymczasowo wyłączonym. Oznacza to, że nie bierze on udziału w propagacji wprzód ani wstecz w danej iteracji (gdyż jego aktywacja ustalana jest na 0).

Zmusza to resztę aktywnych neuronów do kompensowania braku sąsiadów i do samodzielnego wybierania użytecznych cech. Powoduje to, że sieć nie może polegać na pojedynczej drodze. Podczas testowania *Dropout* jest wyłączony.

3.7. Metody balansowania danych

Problem niezbalansowania klas to codzienność w *uczeniu maszynowym*, szczególnie gdy w grę wchodzi medycyna. Przypadki pozytywne (choroba, zgon) , pojawiają się znacznie rzadziej niż przypadki negatywne (zdrowie, przeżycie). I tu zaczynają się schody. Gdy próbujemy przewidzieć *niewydolność serca*, model zwykle skupia się na tej liczniejszej grupie, czyli zdrowych, a zupełnie pomija tych, którzy są naprawdę ważni - chorych. To właśnie ta mniejszościowa klasa ma największe znaczenie z perspektywy lekarzy.

3.7.1. Problem do rozwiązania

Z niezbalansowaniem klas spotykamy się, gdy jedna klasa znacząco przewyższa liczbę obserwacji drugiej klasy. Zbiór z dwiema klasami, które są niezbalansowane, można pokazać jako stosunek:

$$IR = N_{\text{majority}}/N_{\text{minority}}$$

Gdzie N_{majority} to liczba próbek klasy większościowej, a N_{minority} to liczba próbek klasy mniejszościowej. Im wyższe IR , tym większe niezbalansowanie. W medycynie może to sięgać 2:1 a nawet 100:1.

3.7.2. Konsekwencja niezbalansowania

Modele *uczenia maszynowego*, które są trenowane na *niezbalansowanych danych*, mogą:

- Osiągać wysoką dokładność (*Accuracy*) ale niską czyłość (*Recall*) dla klasy mniejszościowej
- Przewidywania klasy większościowej dla większości przypadków
- Słabego wychwytywania wzorców klasy mniejszościowej

W kontekście medycyny, lepiej jest klasyfikować pacjenta niskiego ryzyka jako wysokiego ryzyka, dlatego bilans jest bardzo istotny.

3.7.3. Metody bilansowania:

Używa się trzech głównych podejść, aby uporać się z niezbilansowaniem klas: metody na poziomie danych (data-level methods), metody na poziomie algorytmu (algorithm-level methods) oraz metody hybrydowe. W niniejszej pracy skupimy się na metodach na poziomie danych, które są najczęściej stosowane w praktyce.

3.7.4. Undersampling

Polega na redukcji próbek u klasy, która ma więcej rekordów i zrównuje ją do liczby próbek klasy mniejszościowej. Najprostsza technika to *Random Undersampling*, polegający na losowemu usuwaniu próbek do momenty osiągnięcia pożądanego efektu.

Algorytm Random Undersampling:

Krok 1: Identyfikacja klasy większościową i klasę mniejszościową w zbiorze treningowym.

Krok 2: Obliczyć liczbę próbek do usunięcia:

$$N_{\text{remove}} = N_{\text{majority}} - k * N_{\text{minority}},$$

gdzie k to pożądaný stosunek (np. $k=1$ dla pełnej równowagi).

Krok 3: Losowo wybierać N_{remove} próbek z klasy większościowej i usuń je ze zbioru treningowego.

Krok 4: Trenować model na zbalansowanym zbiorze danych.

Zalety:

- Prosta implementacja
- Mniejszy zbiór treningowy, czyli szybszy czas trenowania
- Zmniejszenie wymagań pamięciowych

Wady:

- Utrata potencjalnie użytecznych informacji zawartych w usuniętych próbkach
- Szansa na niedostateczną reprezentację klasy większościowej
- Ryzyko *underfitting*, w przypadku usunięcia zbyt dużej ilości próbek

3.7.5. Oversampling

Polega na zwiększeniu ilości próbek w klasie mniejszościowej, za pomocą klonowania istniejących próbek. Najprostszą techniką jest *Random Oversampling*, losowo duplikujący klasę mniejszościową

Algorytm Random Oversampling:

Krok 1: Identyfikacja klasy mniejszościową w zbiorze treningowym.

Krok 2: Obliczenie liczby próbek do dodania:

$$N_{\text{add}} = k * N_{\text{majority}} - N_{\text{minority}}$$

gdzie k to pożądany stosunek.

Krok 3: Losowe wybranie N_{add} próbek z klasy mniejszościowej (z powtórzeniami) i dodanie ich kopii do zbioru treningowego.

Krok 4: Trenowanie model na zbalansowanym zbiorze danych.

Zalety:

- Nie ma utraty informacji
- Prosta implementacja

Wady:

- Zwiększa to rozmiar zbioru treningowego (wydłużony czas treningu)
- Może dojść do przeuczenia (overfitting) – model zapamiętuje duplikaty

3.7.6. SMOTE

Stworzył go Chawle i jego współpracownicy w 2002 roku [19]. Jest jedną z najpopularniejszych oraz najskuteczniejszych technik bilansu danych. On również zwiększa liczbę danych w klasie mniejszościowej, ale generując nowe. Są to syntetyczne próbki stworzone poprzez interpolację między istniejącymi próbkami.

Algorytm SMOTE:

Krok 1: Dla każdej próbki x_i z klasy mniejszościowej:

Krok 1.1: Znajdź k najbliższych sąsiadów tej próbki (również z klasy mniejszościowej) w przestrzeni cech, używając metryki odległości (np. odległość euklidesowa).

Krok 1.2: Losowo wybierz jednego z k sąsiadów, oznacz go jako x_{nn} .

Krok 1.3: Wygeneruj nową syntetyczną próbkę x_{new} poprzez interpolację liniową:

$$x_{new} = x_i + \lambda * (x_{nn} - x_i)$$

gdzie λ jest losową liczbą z przedziału $[0, 1]$.

Krok 1.4: Dodanie x_{new} do zbioru treningowego.

Krok 2 : Powtarzaj krok 1 aż do osiągnięcia pożądanej liczby syntetycznych próbek.

Wartość k zazwyczaj jest równa 5 . Parametr λ zajmuje się kontrolowaniem, gdzie wzdłuż odcinka łączącego x_i oraz x_{nn} zostanie umiejscowiona nowa próbka. Jeśli λ wynosi 0, to nowa próbka jest identyczna z x_i , a gdy 1 jest identyczna z x_{nn} , dla wartości pośrednich nowa próbka leży pomiędzy.

Zalety:

- Generowanie nowych próbek, które są zróżnicowane i nie są duplikatami
- Zmniejszenie ryzyka przeuczenia (w zestawieniu z *Oversampling*)
- Skuteczny w zastosowaniach praktycznych

Wady:

- Jest szansa, że wygenerowana próbka będzie w obszarze nakładania się klas (co zwiększa szum)
- Nie uwzględnia rozkładu klasy większościowej przy generowaniu nowych próbek
- Zwiększenie rozmiaru zbioru treningowego

4. Metodologia i środowisko badawcze

Podstawa do tego badania, stanowi publiczny zbiór danych, opublikowany przez Ahmad T. [14] oraz jego współpracowników w 2017 roku w czasopiśmie *PLOS ONE*. Zbiór posiada 299 anonimowych rekordów pacjentów, którzy mają stwierdzoną *niewydolność serca*. Opis zawiera 13 atrybutów (12 cech wejściowych oraz 1 zmienna celu).

Cechy opisujące pacjentów można podzielić na:

- *Cechy demograficzne i kliniczne*: „age” (wiek), „sex” (płeć), „smoking” (palący), „anaemia” (niedokrwistość), „diabetes” (cukrzyca), „high_blood_pressure” (nadciśnienie).
- *Cechy laboratoryjne i fizjologiczne*: „creatinine_phosphokinase” (kinaza kreatynowa), „ejection_fraction” (frakcja wyrzutowa lewej komory), „platelets” (płytki krwi), „serum_creatinine” (kreatynina w surowicy), „serum_sodium” (sód w surowicy).
- *Cecha administracyjna*: „time” (okres obserwacji w dniach).

W związku ze wstępną walidacją, można stwierdzić, że dane nie zawierają brakujących wartości, dzięki czemu można go wykorzystać w takiej formie w dalszych eksperymentach.

6.3. Cel i analiza wstępna

Głównym celem, z którym zmierzali się mechanizmy *uczenia maszynowego*, było binarne klasyfikowanie ryzyka zgonu. Zmienna celu „DEATH_EVENT”, przyjmuje wartości:

- „0”: pacjent przeżył okres obserwacji
- „1”: pacjent zmarł w okresie obserwacji

Analiza rozkładu zmiennej, wykazała niebilansowanie klas (osób których przeżyło jest prawie dwa razy więcej).

Tabela poniżej przedstawia statystyki opisowe dla kluczowych cech numerycznych, które zostały zidentyfikowane w EDA (Eksploracyjnej analizie danych).

Cecha	Średnia	Odchylenie std	Minimum	Mediana	Maksimum
Wiek [lata]	60.83	11.89	40.00	60.00	95.00
Frakcja wyrzutowa [%]	38.08	11.83	14.00	38.00	80.00
Kreatynina [mg/dL]	1.39	1.03	0.50	1.10	9.40
Sód [mEq/L]	136.63	4.41	113.00	137.00	148.00
Płytki krwi [/mL]	263358.03	97804.24	25100	262000	850000
Kinaza kreatynowa [mcg/L]	581.84	970.29	23.00	250.00	7861.00

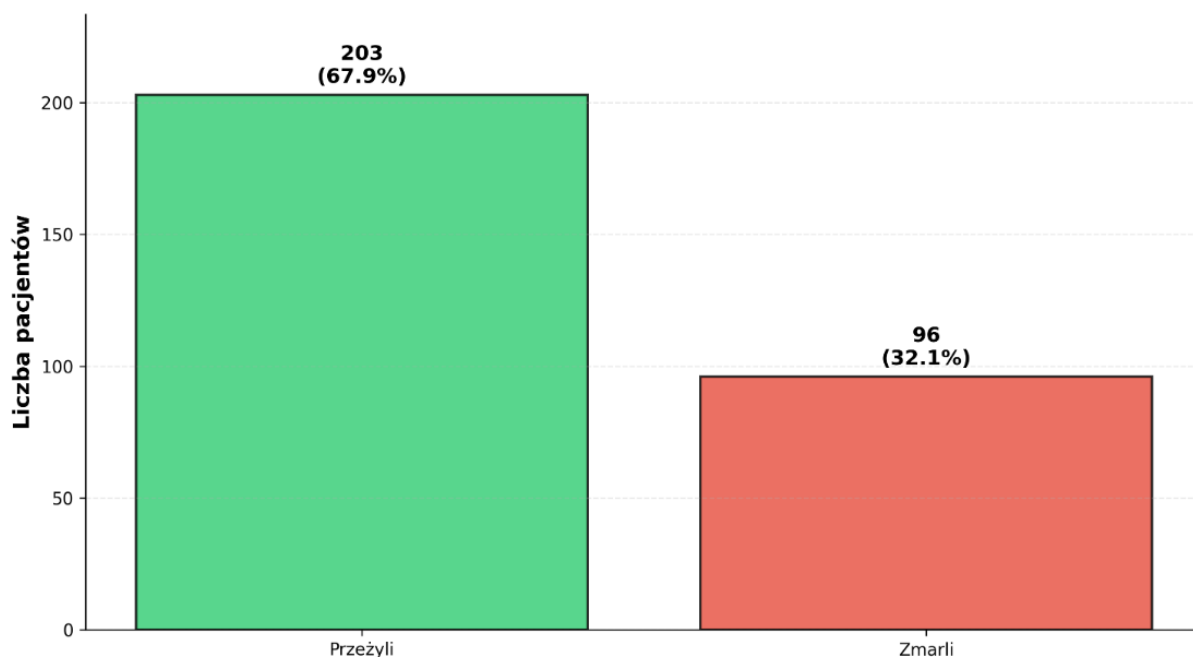
Źródło: opracowanie własne

Tabela pokazuje podstawowe statystyki dla sześciu kluczowych (i numerycznych) cech. Średni wiek pacjentów wynosi 60,83 lat z odchyleniem 11,89 lat w zakresie od 40 lat do 95 lat, co mówi o populacji w średnim lub podeszłym wieku. Ważny wskaźnik wydolności serca, „Fracja wyrzutowa” (lewej komory), której mediana wynosi 38% co wskazuje na problemy ze skurczaniem się lewej komory u pacjentów. „Kreatynina” w surowicy (kryterium związane z nerkami), wyniosła średnio 1,39mg/dL. Warto zwrócić uwagę na wysokie odchylenie standardowe w tej ceście 1,03, co wskazuje na dużą rozbieżność, jeśli chodzi o stan nerek u pacjentów. Średnie stężenie „sód” w surowicy to 136,63mEq/L. Analizując pozostałe cechy można zauważyć, że dane mają dużą rozpiętość, gdzie ekstremalnie wysokie pojedyncze wartości podnoszą średnią.

6.3. Eksploracyjna analiza Danych (EDA)

Eksploracyjna analiza danych (*Exploratory Data Analysis*) stanowi ważny etap każdego projektu. Głównym celem było przeprowadzenie kompleksowego rozpoznania cech badanych, aby lepiej przewidywać „DEATH_EVENT”.

Rozkład zmiennej celu (DEATH_EVENT) w zbiorze danych

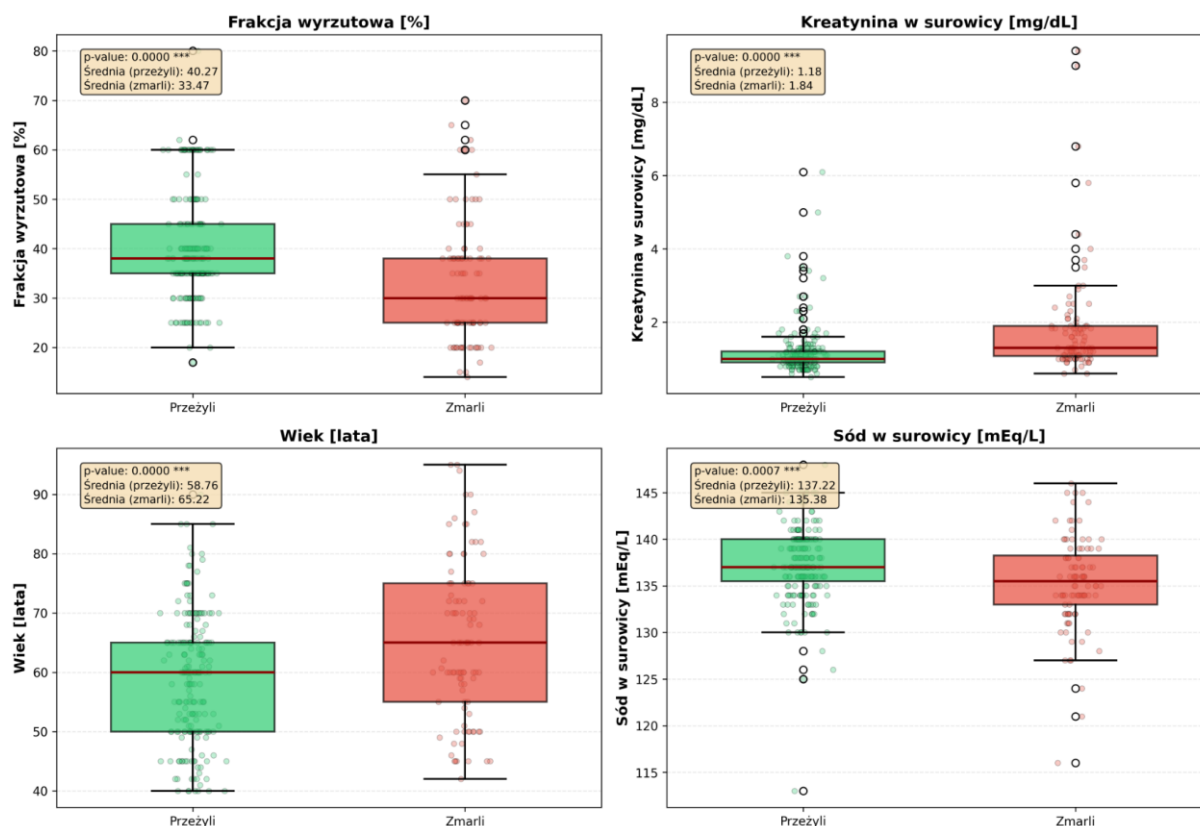


Źródło: opracowanie własne

Na tabeli powyżej wyraźnie widać niezbilansowanie klas. W badaniu wzięło udział 299 pacjentów (203 (67,9 %) przeżyli oraz 96 (32,1%) zmarli). Osób, które przeżyło jest prawie 2 razy więcej. Powoduje to problemy dla modelowania. Model, nauczony na takich danych, osiągnąłby prawie 68% dokładności (*accuracy*) ale jednocześnie będąc bardzo słabym modelem predykcyjnym dla nowych danych. Ważne jest, aby w ocenianiu modelu brać też pod uwagę takie miary odporne na niezbilansowanie klas, takie jak: czułość (*recall*), precyzja (*precision*) i *F1-score*.

Do wskazania najsilniejszych cech związanych z rokowaniem, porównano rozkłady najważniejszych parametrów (na rysunku poniżej)

Porównanie kluczowych cech klinicznych: Przeżyli vs Zmarli



Źródło: opracowanie własne

Analiza wykresów pudełkowych oraz testami istotności statystycznej (t-Student dla cech numerycznych), wykazała ważne różnice między cechami:

- „ejection_fraction” (frakcja wyrzutowa): Osoby poddające się badaniu posiadały niższą medianę oraz rozkład wartości frakcji wyrzutowej, co oznacza, że poważnie uszkodzony mięsień sercowy.
- „serum_creatinine” (kreatynina w surowicy): U zmarłych znaleziono wyższe stężenie kreatyny, co wskazuje na zmniejszenie pracy nerek.
- „age” (wiek): Zmarli pacjenci byli znacząco starsi.
- serum_sodium (sód w surowicy): U zmarłych wykryto mniejszą ilość sodu w surowicy, który jest czynnikiem w *niewydolnościach sercowych*

6.3.3. Wykrycie i eliminacja wycieku informacji (Target leaking)

Ważnym wnioskiem było zauważenie, że jeśli nie wyeliminujemy jednego z cech, to dojdzie do wycieku informacji. Okazało się, że „time” (czas obserwacji) wykazywał niesamowicie silną korelację ze zmienną „DEATH_EVENT”. W związku z tym cecha „time” musiała zostać

wykluczona z *uczenia* modeli. Niewykluczenie go, można by było nazwać oszustwem, ponieważ na początku ani w środku nie znamy czasu, który pozostał pacjentowi do zgonu czy końca obserwacji. Bez wyłączenia cechy „time”, model *uczenia maszynowego* zobaczyłby pacjenta z krótkim czasem badania i uznałby go za martwego, mimo że żyje tylko dopiero zaczął obserwację.

6.3.3. Metryki oceny modeli

Podczas klasyfikacji, sama trafność (accuracy), czyli ilość poprawnie zidentyfikowanych próbek, jest niewystarczająca a nawet często myląca, szczególnie przy *niezbilansowanych danych* (takich jakie analizuję). Model mógłby przewidywać zawsze klasę, w której znajdują się większość i wciąż posiadać 68% accuracy. Dlatego stworzono zestaw metryk, które oceniają model z różnych perspektyw. Dla klasyfikacji binarnej, gdzie klasę pozytywną, czyli zgon oznacza „1”, przedstawia się 4 wartości:

- Prawdziwie pozytywne (True Positive, TP): Pacjent zmarł i model poprawnie to przewidział.
- Fałszywie pozytywne (False Positive, FP): Pacjent przeżył, ale model błędnie zaklasyfikował go jako zmarłego („fałszywy alarm”).
- Prawdziwie negatywne (True Negative, TN): Pacjent przeżył i model poprawnie to przewidział.
- Fałszywie negatywne (False Negative, FN): Pacjent zmarł, ale model błędnie zaklasyfikował go jako przeżywającego („przegapiony przypadek”).

Wartości te umożliwiają zrobienie obliczeń miar:

- Precyzja (*Precision*) – mierzy wiarygodność modelu. Mówi o tym ilu pacjentów, które zostały oznaczone przez model jako zgon, rzeczywiście zmarło:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Wynika z tego, że jeśli model wykryje zagrożenie, można mu ufać

- Czułość (*Recall*) – Pokazuje liczbę, opisującą spośród wszystkich pacjentów, którzy zmarli, a model zakwalifikował ich jako zmarłych:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

W medycynie bardzo istotna miara. Im wyższa czułość tym mniej przeoczonych pacjentów, którzy są zagrożeni (warto mieć fałszywy alarm niż pominąć jakieś zagrożenie)

- *F1-score* – Stanowi miernik ogólnej skuteczności modelu. Istotny przy *niezbilansowanych danych*, ponieważ znajduje kompromis między precyzją a czułością

$$F1_score = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Mieści się w zakresie od 0 do 1, w którym 1 oznacza model idealny

- *AUC-ROC* (*krzywa ROC*) – Ilustruje zależność między czułością a liczbą fałszywych alarmów (False positive). Pole pod krzywą oznacza:

- $AUC = 0,5$ – to samo co rzut monetą
- $AUC = 1,0$ – model doskonale separuje klasy

6.3.3. Środowisko pracy

Wszystkie etapy projektu zostały zrobione w języku Python(wersji 3.11). Został wybrany ze względu na to, że podobno stanowi standard w dziedzinie *uczenia maszynowego* oraz analizy danych dzięki dużej ilości bibliotek. Kluczowe biblioteki w projekcie:

- *Pandas* (wersja 1.5.3) i *Numpy* (wersja 1.24.3) – do obróbki i operacjach na danych.
- *Scikit-learn* (wersja 1.3.0) – do *preprocessingu* danych (np. *StandardScaler*), podziału na zbiory, implementacji modelu *RandomForest* oraz obliczania metryk.
- *TensorFlow* (wersja 2.12.0) – do budowania, trenowania i ewaluacji sieci nauronowych (w tym wielowarstwowego perceptronu (MLP))
- *Matplotlib* (wersja 3.7.1), *Seaborn* (wersja 0.12.2) i Excel – do tworzenia wizualizacji

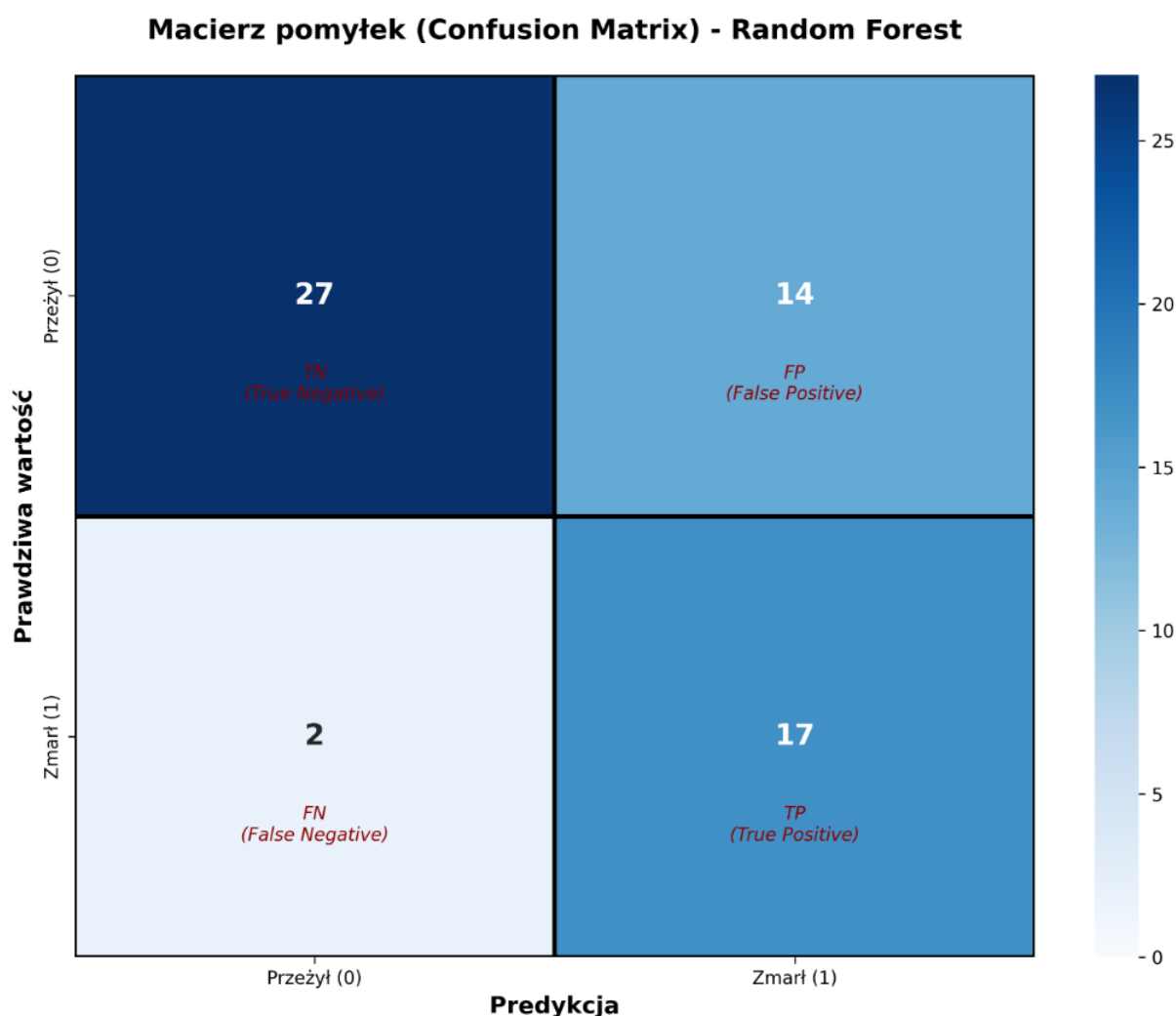
5. Eksperymenty i wyniki

Ten rozdział stanowi główną część pracy. Przedstawia zarówno eksperymenty jak i wyniki. Jego struktura pokazuje podejście badawcze od budowy modelu bazowego, przez próby ulepszenia, aż po eksperymenty ze sztucznymi sieciami neuronowymi.

5.1. Eksperyment 1: Model Bazowy Random Forest

Jako pierwszy krok należało stworzyć model bazowy, który będę traktował jako odniesienie do dalszych eksperymentów. Jako model bazowy wybrano Random Forest (zaimplementowany za pomocą biblioteki „Scikit-learn”, ponieważ pojawia się w oryginalnej pracy oraz jest odporny na przeuczenia. Model został wytrenowany na trzech głównych cechach: „age”, „ejecion_fraction” oraz „serum_creatinine”. Zostało użyte „RandomizedSearchCV” do walidacji krzyżowej, aby znaleźć jak najlepsze hiper parametry. Wyniki przedstawione są w tabeli oraz rysunku poniżej

Metryka	Wartość
F1-score	0,680
Recall (czułość)	0,895
Precision (precyzja)	0,588
AUC-ROC	0,769



Źródło: opracowanie własne

Przedstawia on macierz pomyłek dla modelu *Random Forest*, gdzie zidentyfikował 17 z 19 pacjentów, którzy umarli, co z medycznego punktu widzenia jest pożądanym wynikiem. Jest to prawie 90% dobrze zidentyfikowanych osób w grupie wysokiego ryzyka.

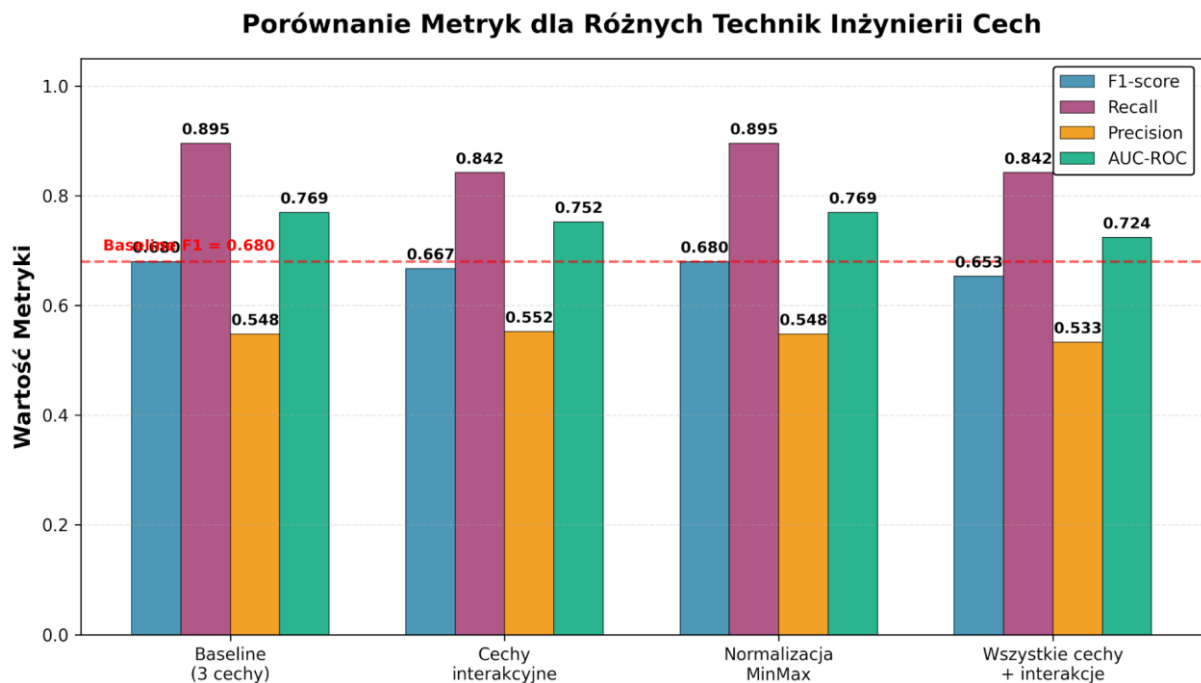
Wniosek: Model bazowy pokazuje bardzo dobre wyniki w kontekście medycznym.

5.2. Eksperyment 2: Model Random Forest z Inżynierią cech

Następnie zajęto się poprawą wyników modelu bazowego za pomocą technik inżynierii cech. Eksperymenty, testowały wpływ:

- Cechy interaktywne: stworzenie nowych cech, które są połączeniem istniejących cech (*age * serum_creatinine*, *ejection_fraction * serum_sodium*, *age * ejection_fraction*)
- *MinMaxScaler*: sprawdzenie czy zmiana metod skalowania danych ma wpływ na wyniki. Zamiast *StandardScaler* (średnia = 0, odchylenie standardowe = 1), wykorzystano *MaxScaler*, który skaluje dane do zakresu [0,1]

- Użycia wszystkich dostępnych cech, które dostarczyły dane

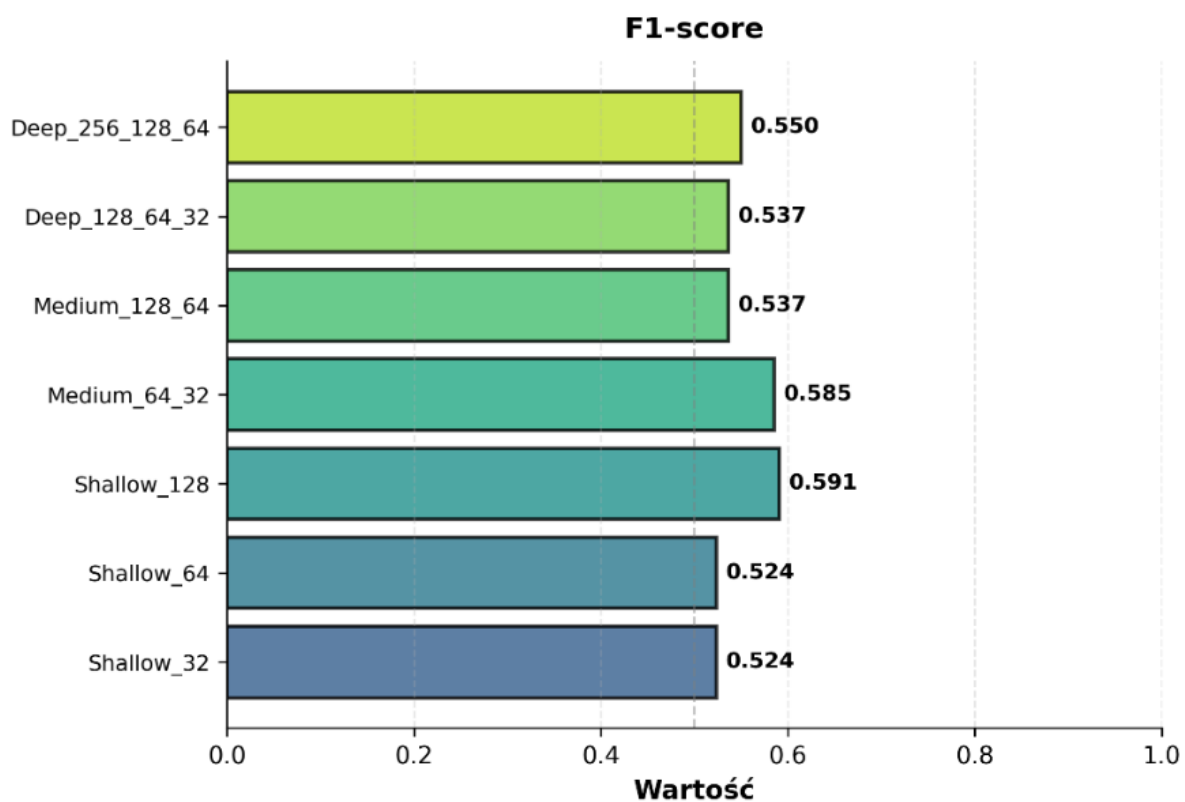


Źródło: opracowanie własne

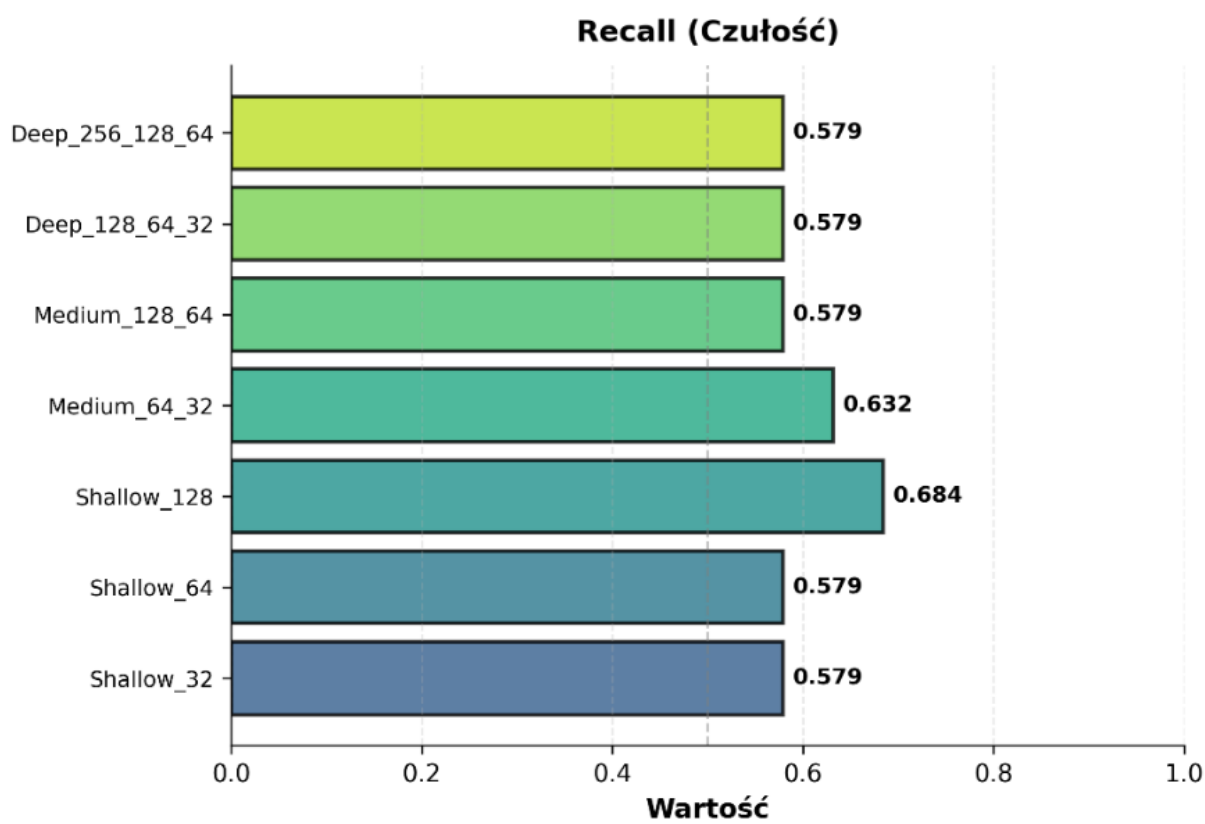
Wniosek: Na można zauważyć, że żadne z modyfikacji, nie spowodowało polepszenia się wyników modelu, minimalny zbiór cech okazał się być najlepszą drogą.

5.3 Eksperyment 3: Model sieci neuronowej na danych niezbalansowanych

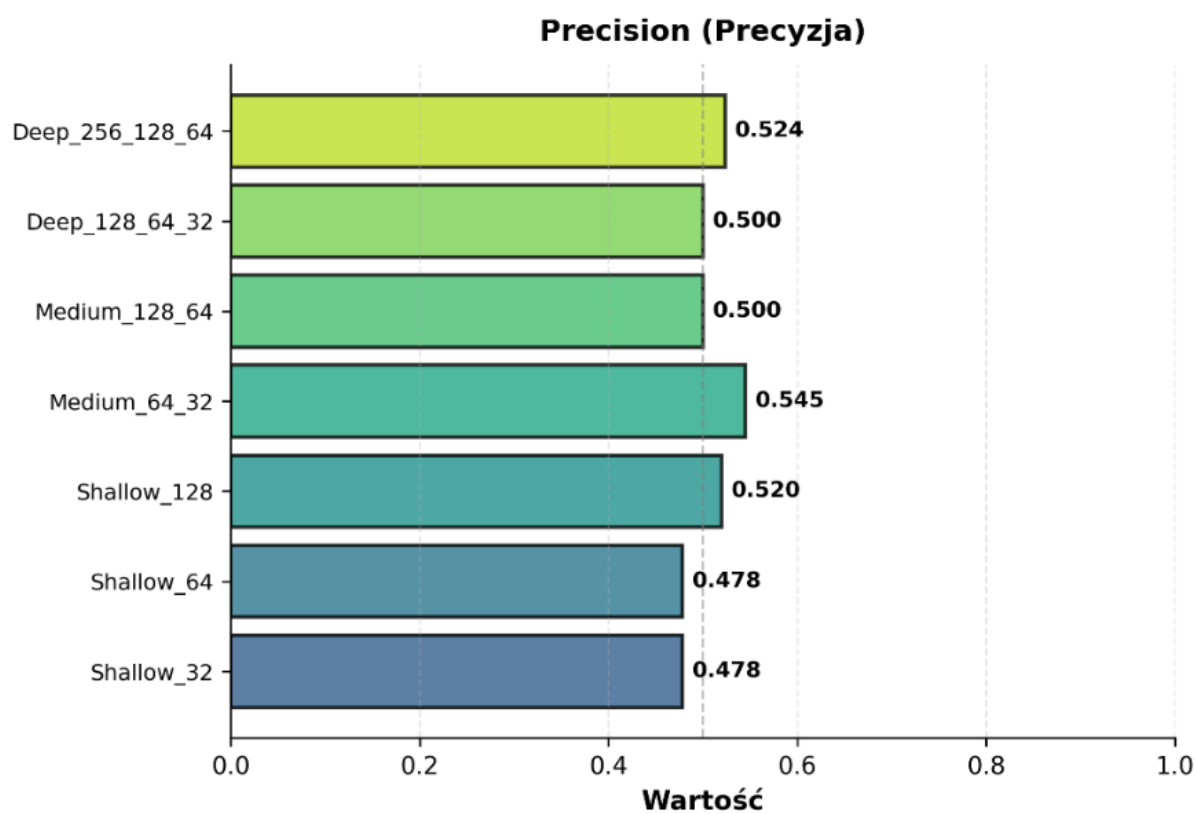
Ten etap poświęcono na eksploracji wielowarstwowego perceptronu (MLP). Celem badania było sprawdzenie czy model, który opera się na głębokim uczeniu potrafi przewyższyć klasyczny algorytm Random Forest.



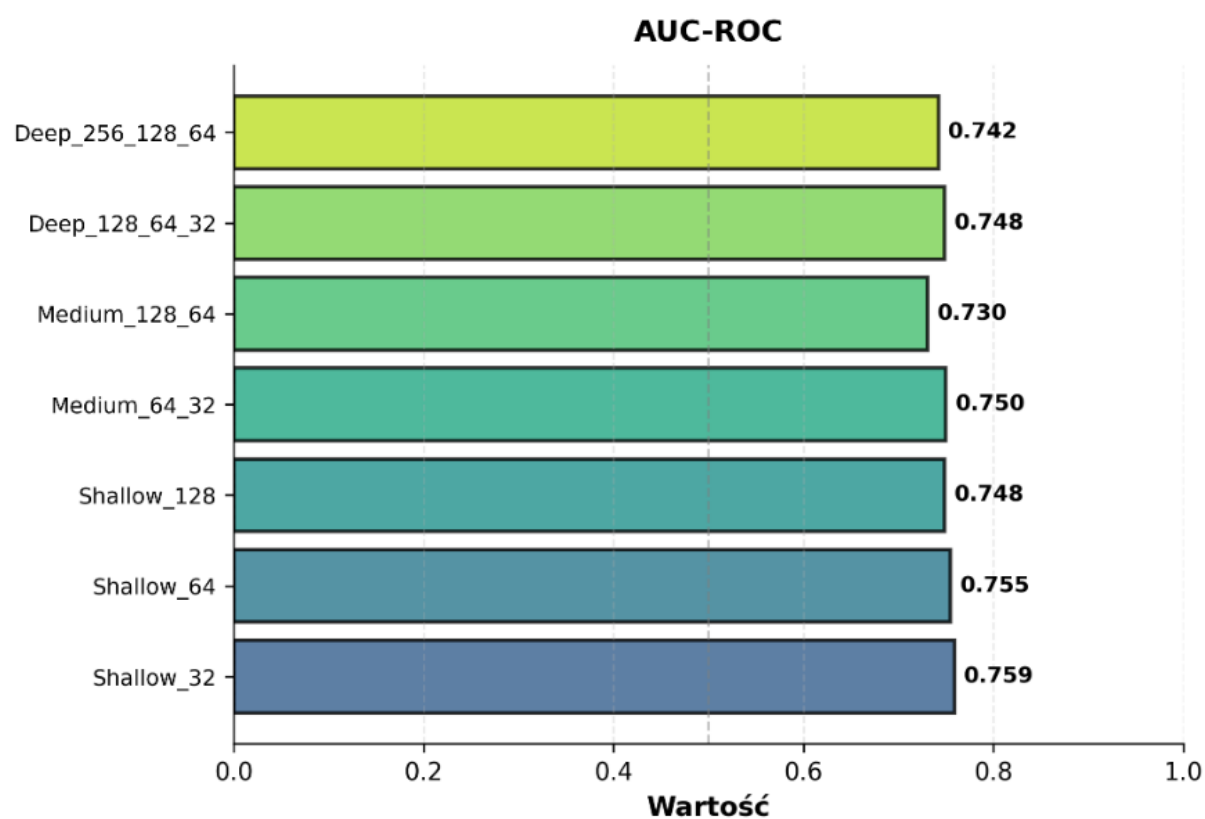
Źródło: opracowanie własne



Źródło: opracowanie własne



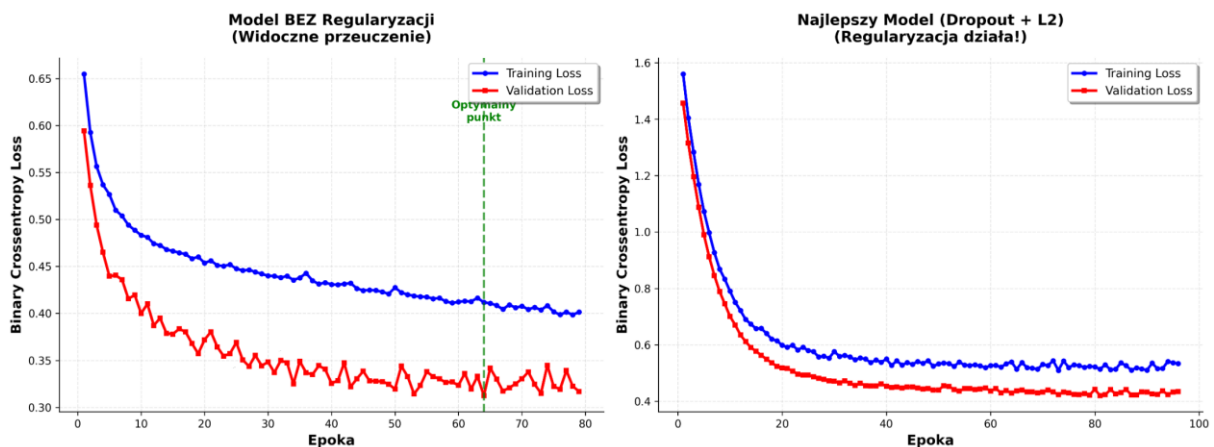
Źródło: opracowanie własne



Źródło: opracowanie własne

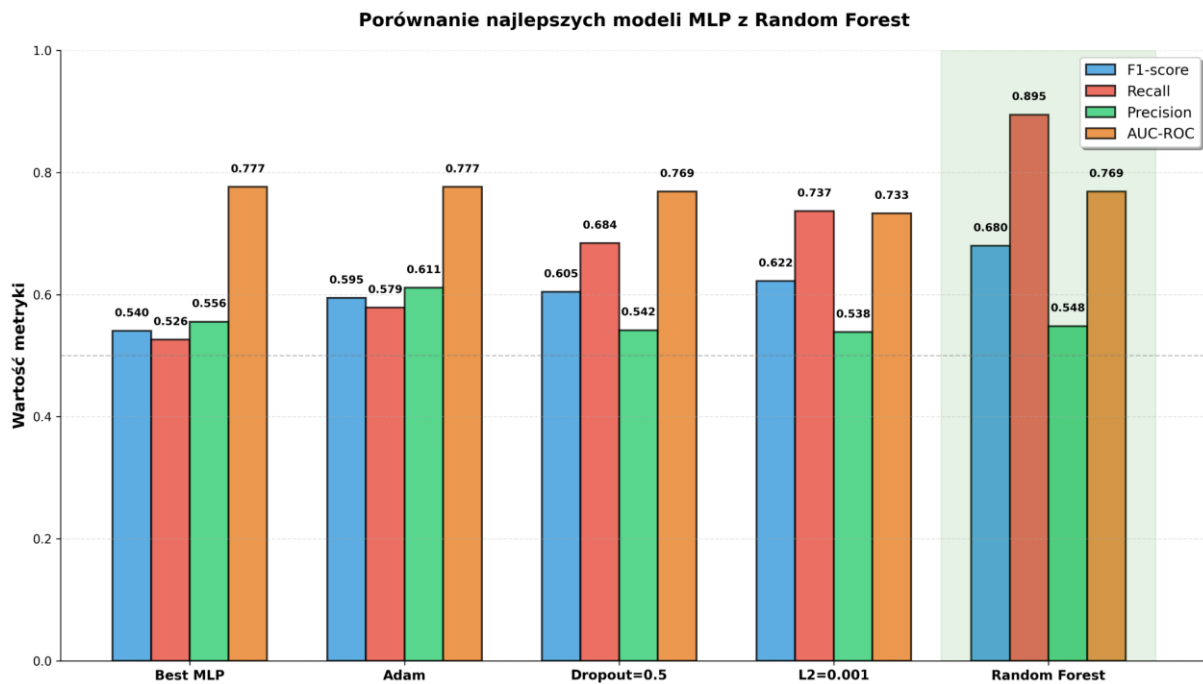
Porównanie metryk dla różnych głębokości oraz ilości neuronów. Liczby po słowach *Shallow*, *Medium* i *Deep* przedstawiają ilość neuronów a ilość liczb głębokość (np. *Shallow_32* pokazują jedną warstwę i 32 neurony, natomiast *Medium_64_32* dwie warstwy z 32 oraz 64 neuronami). Z wyniku można wywnioskować, że duża głębokość sieci nie powoduje lepszych wyników, a najbardziej optymalną jest jedna warstwa z 128 neuronami.

Przeprowadzone eksperymenty, w których testowano wpływ zmiany parametrów na wydajność sieci, takie jak: architekturę sieci, funkcję aktywacji, techniki regularyzacji (*Dropout* i *L2*) i optymalizatory.



Źródło: opracowanie własne

Wnioski: Analiza krzywych uczenia, pokazują, że regularyzacja była bardzo istotna pod kątem przeuczenia (zapobiega przeuczeniu), na *małym zbiorze danych*. Dzięki temu model uczy się stabilnie.



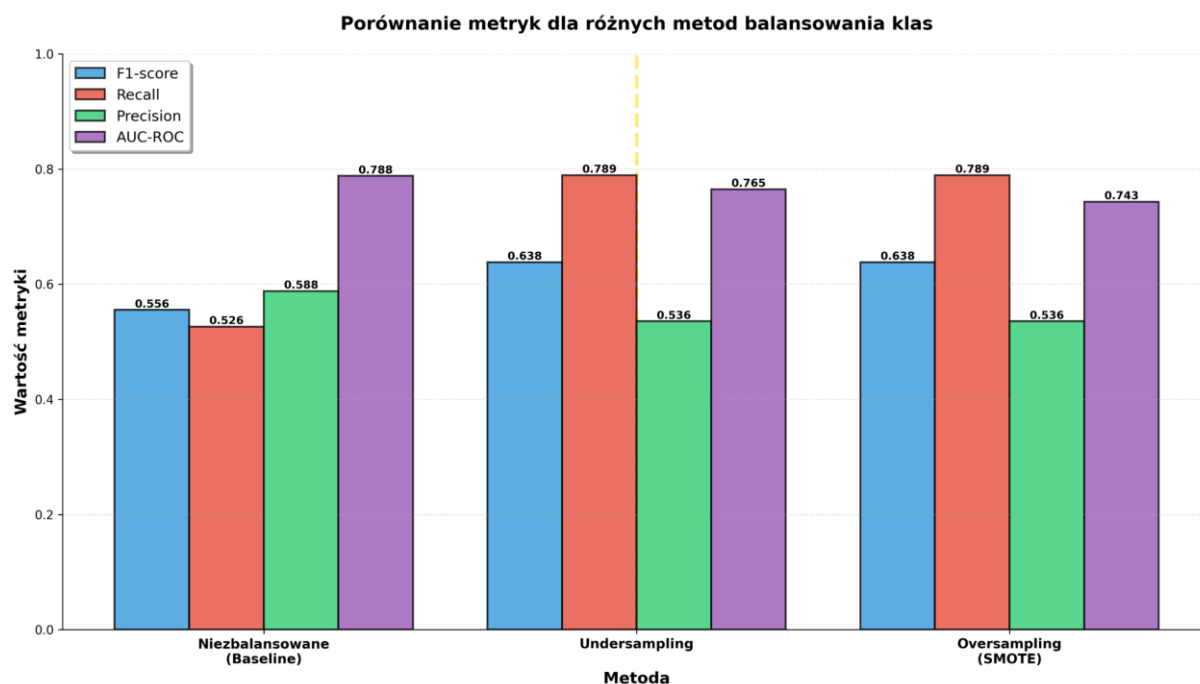
Źródło: opracowanie własne

Wnioski: Na wykresie powyżej można zauważyć, że *Best MLP* (najlepszy model *MLP*), mimo optymalizacji, nie był w stanie przebić *Random Forest* na danych niezbalansowanych.

5.4. Eksperyment 4: Model sieci neuronowej na danych zbalansowanych

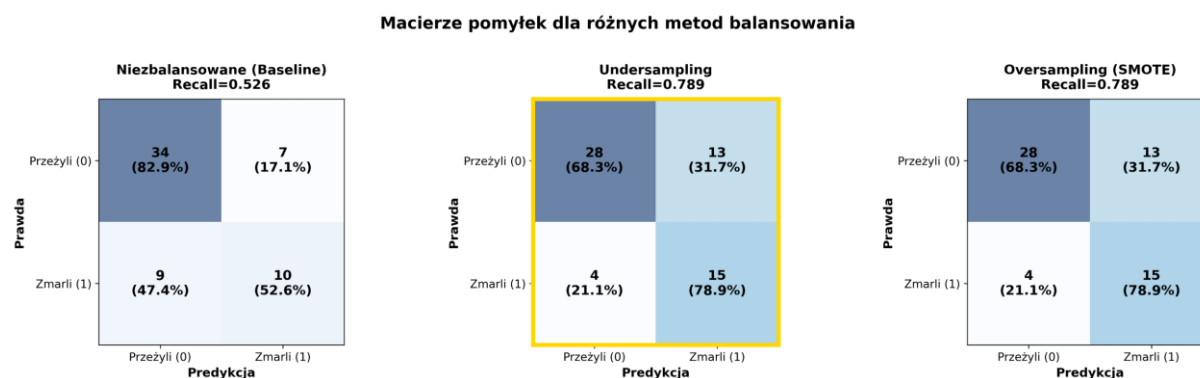
Niskie wyniki modelu *MLP*, może pokazywać, że sieć jest wrażliwa na *niezbalansowane dane*. W tej części skupię się na tym, czy bilans klas jest w stanie poprawić wyniki sieci neuronowej. Użyte metody:

- *Undersampling*: polega na losowym usunięciu wierszy z większej klasy.
- *SMOTE*: generowanie próbek dla mniejszej klasy.



Źródło: opracowanie własne

Wniosek: Wyniki wykresu powyżej, są bardzo ważne, ponieważ obie techniki spowodowały polepszenie się wyników, a w szczególności metryki *Recall* (która z medycznego punktu widzenia jest niezmiernie istotna). Metryka *F1-score* również uległa widocznemu podwyższeniu, co potwierdza, że zbilansowanie danych to był dobry pomysł, można również to zaobserwować na macierzy pomyłek na obrazie poniżej.



Źródło: opracowanie własne

Bardzo interesujące jest również to, że metoda „Undersampling” dała prawie identyczne wyniki jak bardziej złożone *SMOTE*, mimo że *MLP* miało w tym przypadku dostęp do jeszcze bardziej ograniczonego zbioru. *MLP* po zastosowaniu *Undersampling* z wynikiem *F1-score* równym 0,638, stał się najlepszym modelem sieci neuronowych, ale warto jest podkreślić, że nie przebił *Random Forest*.

6. Dyskusja i wnioski

6.1 Dyskusja wyników

Przeprowadzone eksperymenty pozwoliły zaobserwować kilka istotnych wniosków, jeśli chodzi o *predykcję śmiertelności w niewydolnościach serca* na badanym zbiorze danych.

Random Forest istniejący w pracy jako model bazowy: Pierwszym co można zauważyć, jest fakt, że mimo prostoty, *Random Forest* był w stanie osiągnąć wysoki wynik wydajności. Szczególnie należy zwrócić uwagę, na jego umiejętności predykcyjne u grupy osób wysokiego ryzyka, gdzie *Recall* wynosił 89,5%, co spowodowało problemy u sieci neuronowej w przebiciu tego wyniku. Oznacza to, że w dziedzinie *uczenia maszynowego*, dobór odpowiedniego modelu oraz odpowiednich cech jest bardziej efektywny niż skomplikowane modele.

Inżynieria cech okazała się nie być bardzo efektywna. Inne cechy interakcyjne nie spowodowały polepszenia wyników, a jednocześnie dyskretyzacja obniżyła wyniki. Wskazuje to na to, że *Random Forest* poradził sobie lepiej w nakreśleniu nieliniowych zależności, a próby modelowania poskutkowały utratą wyników oraz wprowadzenie szumu.

Wrażliwość sieci neuronowych na *niezbilansowane dane*. Eksperyment z sieciami neuronowymi (*MLP*), pokazał jak istotne jest zbilansowanie klas. Model trenowany na niezbilansowanych danych, pomimo optymalizacji, nie dawał sobie rady ze skutecznym rozpoznawaniem mniejszej klasy (przypadku zgonu). *Recall* w tym przypadku wynosił 52,6%. Dopiero zastosowanie bilansu klas sprawiło, że czułość się poprawiła i osiągnęła poziom 78,9%. Wniosek jaki warto z tego wyciągnąć, brzmi, że w przypadku sieci neuronowych, nie jest ważna sama architektura sieci, ale i odpowiednie przygotowanie danych.

Bardziej skomplikowane modele nie zawsze są lepsze niż te proste. Porównując wyniki eksperymentów, okazało się, że prostsze rozwiązania okazały się skuteczniejsze. Jednocześnie *Undersampling* okazał się być skuteczniejszy niż *SMOTE*, mimo że jest prostszy. Oznacza to, że w budowie najlepszego modelu ważne jest, aby pamiętać o najprostszych i sprawdzonych metodach.

6.2. Wnioski końcowe

Bazując na przeprowadzonych badaniach, można skonstruować następujące wnioski:

1. Najwyższą skuteczność w *predykcji śmiertelności pacjentów z niewydolnością serca* na otrzymanym zbiorze badawczym (składającym się z 299 pacjentów), osiągnął model *Random Forest* z wynikami: *F1-score* = 68% oraz *Recall* = 89,5%.
2. Zastosowanie zaawansowanych technik inżynierii cech, nie spowodowało poprawienia jakości modelu.
3. *MLP* okazał się być bardzo wrażliwy na niezbilansowane klasy. Dopiero ich zbilansowanie pozwoliło na poprawę wyników modelu, a szczególnie na *Recall* które znacznie wzrosło.
4. Praca pokazuje, że klasyczne algorytmy *uczenia maszynowego*, w połączeniu z odpowiednim dobraniem cech, które model bierze pod uwagę podczas predykcji, wciąż stanowi konkurencję dla głęboko uczących się modeli.

6.3. Ograniczenia i kierunek rozwoju

Ważne jest, aby mieć świadomość z ograniczeń jakie badanie na mnie narzuciło.

Najistotniejszym był *mały zbiór danych*, wynoszący jedynie 299 pacjentów. Na takich zbiorach, sieci neuronowe mają problem z generalizacją i są mniej odporne na przeuczenie. Moim zdaniem to jest główny czynnik, który wpłynął na niskie wyniki *MLP*.

Biorąc to pod uwagę, oto moje propozycję dalszego rozwoju:

- Przeprowadzenie eksperymentu na większej ilości danych, co pomogłoby na określenie czy wnioski z mojej pracy są uniwersalne.
- Wdrożenie innych algorytmów do badania, mogłoby skutkować dostaniem jeszcze lepszych wyników.
- Zaawansowane Techniki Interpretowalności pomogłoby w głębszym wglądzie na proces decyzyjny (szczególnie jeśli chodzi o sieci neuronowe).
- Zestawienie wyników sieci z wynikami lekarzy, co mogłoby pokazać, na jakim poziomie obecnie znajduje się sieć

Bibliografia

- [1] World Health Organization. (2022, May 25), **Cardiovascular diseases**. <https://www.who.int/health-topics/cardiovascular-diseases>
- [2] Savarese G., Lund L.H. (2017), *Global public health burden of heart failure*, **Cardiac Failure Review**.
- [3] Benjamin E.J. et al. (2019), *Heart Disease and Stroke Statistics*, **Circulation**.
- [4] Ponikowski P. et al. (2014), *Heart failure: preventing disease and death worldwide*, **ESC Heart Failure**.
- [5] Hannun A.Y. et al. (2019), *Cardiologist-level arrhythmia detection with convolutional neural networks*, **Nature Medicine**.
- [6] Bernard O. et al. (2018), *Deep learning techniques for automatic MRI cardiac segmentation*, **IEEE Transactions on Medical Imaging**.
- [7] Krittanawong C. et al. (2017), *Machine learning prediction in cardiovascular diseases*, **Journal of the American College of Cardiology**.
- [8] Chicco, D., & Jurman, G. (2020). *Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone*, **BMC Medical Informatics and Decision Making**, 20(1), 1-16.
- [9] Breiman, L. (2001). *Random forests*, **Machine Learning**. <https://doi.org/10.1023/A:1010933404324>.
- [10] Nair, V., & Hinton, G. E. (2010). *Rectified linear units improve restricted Boltzmann machines*, **Proceedings of the 27th International Conference on Machine Learning (ICML-10)**.
- [11] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*, **Nature**, 323(6088) <https://gwern.net/doc/ai/nn/1986-rumelhart-2.pdf>.
- [12] Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*, **arXiv preprint arXiv:1412.6980**.
- [13] Srivastava, N. et al. (2014) *Dropout: A simple way to prevent neural networks from overfitting*, **The Journal of Machine Learning Research**, 15(1), pp. 1929-1958.
- [14] Ahmad, T., Munir, A., Bhatti, S. H., Aftab, M., & Raza, M. A. (2017). *Survival analysis of heart failure patients: a case study*, **PloS one**, 12(7).
- [15] Breiman, L. (1996). *Bagging Predictors*. **Machine Learning**, 24(2), 123–140.
- [16] Freund, Y., & Schapire, R. E. (1997). *A decision-theoretic generalization of on-line learning and an application to boosting*, **Journal of Computer and System Sciences**, 55(1), 119–139.
- [17] Friedman, J. H. (2001). *Greedy function approximation: A gradient boosting machine*. **The Annals of Statistics**, 29(5), 1189–1232.
- [18] Kim, Y., Kim, Y. M., Choi, H. W., & Kang, C. (2023). *Ensemble machine learning of gradient boosting (XGBoost, LightGBM, CatBoost) and attention-based CNN-LSTM for snake venom classification*, **Toxins**, 15*(3), 171.

[19] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*, **Journal of Artificial Intelligence Research**, 16, 321–357.

Link do GitHuba:

https://github.com/krzowskiartur/praca_inzynierska.git